Empirical Software Engineering (2026) 31:118
https://doi.org/10.1007/s10664-026-10858-8

An evaluation study of large language models for addressing code quality issues

Rares Patcas[1] · **Simona Motogna**[1]

Received: 16 August 2025 / Accepted: 27 March 2026

Abstract
This empirical study investigates how state-of-the-art Large Language Models (LLMs) can automatically resolve code issues identified by SonarQube, a widely used static analysis tool. As automated maintenance becomes more common, combining AI models with rule-based analysis offers a promising approach to improving code quality. We compare six LLMs, including GPT-4o, Gemini 2.0 Flash, Claude 3 Opus, Mistral Large, Grok 3, and Deep-Seek V3, in performing automated code repair. Using a unified prompt strategy, SonarQube issues are mapped into structured prompts, and LLM-generated fixes replace affected functions in the source code. We evaluate repairs based on syntactic correctness, reduction in SonarQube reported issues, and introduce the Static Repair Success Rate (SRSR), a strict metric that measures the proportion of syntactically valid repairs that resolve all original issues without introducing new ones, followed by a semantic analysis to assess whether the repaired code preserved the intended program behavior. Overall, the average reduction in SonarQube-reported issues, calculated across all models and projects, was about 36.02%. The best result for a single project was achieved by the Grok 3 model, which reduced issues by 71.54%. These findings suggest that LLMs can enhance automated refactoring and help reduce static analysis–reported issues. They offer insights for integrating AI into development workflows, helping companies streamline maintenance, reduce technical debt, and sustain high code quality.

Keywords Large language models · Static analysis · SonarQube · Automated code repair

Communicated by: Andrea Stocco.

✉ Rares Patcas
rares.patcas@ubbcluj.ro

Simona Motogna
simona.motogna@ubbcluj.ro

[1] Faculty of Mathematics and Computer Science, Babeş-Bolyai University, Mihail Kogălniceanu, Cluj-Napoca 400084, Cluj, Romania

Published online: 13 April 2026

1 Introduction

Static code analysis tools are an integral part of modern software development workflows. These tools help developers identify potential issues in source code early in the development cycle by enforcing rule-based checks and flagging noncompliant patterns. This practice improves code quality, reduces technical debt, and supports long-term maintainability (Vassallo et al. 2020). The primary focus of static analysis is detection, and the tools provide diagnostic information but usually do not suggest specific fixes. As a result, developers must interpret and address the issues manually, which can be time-consuming and influenced by the development team's experience and coding practices.

Large language models (LLMs) have been applied to various software engineering tasks. Trained on large datasets that include publicly available code and written text, LLMs have demonstrated the ability to recognize programming patterns, generate code snippets, suggest documentation, and perform basic code refactoring. According to Jiang et al. (2024), recent models show multiple capabilities relevant to code maintenance. This study examines wheth-er LLMs can be used to generate context-aware fixes that are compatible with existing codebases and aligned with static analysis rules.

Our evaluation focuses on the ability of LLMs to generate valid and actionable refactorings that repair code issues reported by the static analysis tool SonarQube. We chose this tool because of its ability to detect a wide variety of coding issues, broad language support, and widespread adoption in both industry and research (Quezada Sarmiento et al. 2017). An empirical study (Lenarduzzi et al. 2023) shows that it detects a wider range of issues than several other static analysis tools, making it well-suited for comparative evaluations. While SonarQube, like other static analysis tools, sometimes produces false positives, its ability to uncover diverse and meaningful issues still makes it a realistic and rigorous testbed. Moreover, as highlighted by Lefever et al. (2021), there is a lack of empirical research evaluating automated repair methods using real-world static analysis tools. Together, these considerations motivate our use of SonarQube as the foundation for assessing LLM performance.

In this study, we decided to focus on Python because of its popularity and strong adoption in the software development community. Python is an appropriate choice for use in this software engineering research paper, as it allows us to evaluate a variety of concerns, such as syntax, maintainability, security, and design, ensuring that assessments reflect real-world coding practices and challenges. According to the TIOBE Index[1] (2024), Python has ranked as the most popular programming language for two consecutive years.

The assessment is based on open-source Python projects from the dataset introduced in Moldovan et al. (2024), which contains 80 repositories selected for research on code quality, static analysis, and automated repair. From this dataset, a subset of 20 projects was chosen for detailed evaluation to balance representativeness with analytical feasibility. The selection focused on repositories with higher **fork counts**, reflecting their popularity and community engagement, while also confirming sustained development activity through their **commit histories**. This approach ensured the inclusion of widely used, actively maintained, and diverse Python projects while keeping the analysis tractable and suitable for fine-grained inspection.

The study evaluates the extent to which six LLMs, namely GPT-4o, Gemini 2.0 Flash, Claude 3 Opus, Mistral Large, Grok 3, and DeepSeek V3, can generate fixes aligned with

[1] https://www.tiobe.com/tiobe-index/

SonarQube diagnostics. These models represent diverse architectures, enabling a meaningful comparative analysis and highlighting performance variability. Further details on model selection and prompting strategies are discussed in Sections 4.2 and 4.3.

The paper shows that large language models (LLMs) can address SonarQube findings, but performance varies by model and category, with some outputs fixing issues correctly while others introduce new ones. As shown by Jin et al. (2023), LLMs can generate code, functions, or documentation from natural language prompts, yet automated code repair in response to static analysis diagnostics presents a distinct challenge, requiring precise changes to existing code based on externally defined diagnostic rules. This makes static analysis remediation a rigorous benchmark for evaluating LLM capabilities in practical software maintenance.

2 Theoretical Background

2.1 Static Code Analysis Tools

Static code analysis is an essential technique in software engineering used to identify defects without executing the code. It enables developers to detect issues early in the development process, reducing costs and meeting delivery deadlines. According to the study (Yeboah and Popoola 2023), static analysis is increasingly employed because of its effectiveness in early defect detection. That study evaluated four widely used tools, SonarQube, FindBugs, PMD, and Checkstyle, across 50 open source projects in Java, C, C++, and Python. Based on precision, recall, and F1 score, SonarQube achieved the best performance with a precision of 0.83, a recall of 0.87, and an F1 score of 0.85. The other tools scored lower: FindBugs obtained a precision of 0.78, a recall of 0.82, and an F1 score of 0.80, PMD reached 0.71 precision, 0.76 recall, and 0.73 F1, while Checkstyle recorded 0.69 precision, 0.71 recall, and 0.70 F1. These results highlight SonarQube's superior defect detection, especially for multi-language and complex codebases.

SonarQube is a widely adopted static analysis tool that identifies code quality issues in many programming languages. It analyzes source code against a set of predefined rules and provides immediate feedback and actionable insights without running the code. As explained by Quezada Sarmiento et al. (2017), SonarQube integrates seamlessly into modern development workflows, including IDEs, version control systems, and CI/CD pipelines, making it a practical choice for enforcing coding standards and maintaining long-term software health.

The architecture of SonarQube consists of three main components. The Analyzer scans the code and creates snapshots of its state. The Database stores configurations, metrics, and snapshots to enable tracking and comparison over time. The Server provides a user friendly web interface where developers and managers can review results, adjust configurations, and decide on actions to improve code quality. Together, these components deliver continuous feedback on code health and help teams manage technical debt throughout the development lifecycle.

2.2 Large Language Models for Code Repair

Large Language Models (LLMs) have rapidly become valuable tools in automating software engineering tasks, including the repair of defective or suboptimal code. Among various software maintenance applications, code repair has emerged as a particularly promising

use case, where LLMs assist in generating patches that resolve coding issues (Xia et al. 2023). Complementing these technical efforts, recent work has also begun establishing methodological guidelines for empirical studies involving LLMs in software engineering (Baltes et al. 2025).

Early LLMs like Codex demonstrated strong performance in this domain. Codex, introduced in Chen et al. (2021), was fine-tuned on a large dataset of publicly available code from GitHub and evaluated using the HumanEval benchmark. Although not specifically trained for automated program repair, Codex has shown notable success in generating functionally correct code and resolving small-scale programming faults.

Similarly, CodeGen, an open-access model introduced in Nijkamp et al. (2022), was designed for multi-turn program synthesis and trained on both natural language and code. It has been shown to perform competitively in zero-shot code generation tasks, and its modular architecture makes it amenable to adaptation for repair tasks. More recently, StarCoder (Li et al. 2023), developed by the BigCode project, has extended this approach by focusing on permissively licensed training data, reinforcing the need for transparency and reproducibility in the development of LLMs for code tasks.

In parallel with model development, the research community has begun to consolidate theoretical and empirical knowledge on LLM-driven code repair. A recent survey (Zhang et al. 2023) provides a comprehensive overview of learning-based automated program repair (APR), outlining key approaches, evaluation frameworks, and open challenges. It emphasizes the importance of auxiliary information, such as test cases, static warnings, or execution traces, in guiding the repair process and discusses the limitations of LLMs when used in isolation, including hallucinated fixes, brittleness to prompt phrasing, and generalization issues.

While most LLM-based repair studies have focused on compiler errors or functional bugs, integrating these models into static analysis workflows introduces new challenges and opportunities. Static analysis tools produce structured, localized warnings that are often easier to isolate than runtime faults. However, the fixes required may range from simple one-line changes to broader refactorings, and LLMs must interpret these warnings accurately to produce meaningful edits. Moreover, static analysis warnings are not always definitive indicators of faulty behavior, which adds ambiguity to the repair task and increases the likelihood of semantic drift in the model's output.

3 Related Work

Addressing software quality issues has been a central goal in software engineering research. Recent work has explored automated techniques for improving code, with a growing focus on large language models (LLMs) for generating fixes. In this section, we review studies on automated repair approaches that aim to improve code quality. While some target a broad spectrum of issues, others concentrate on specific problem types such as bug reports, security vulnerabilities, or static analysis violations. Even when focused narrowly, these efforts contribute to the overall goal of improving software quality by removing defects, reducing technical debt, or enhancing maintainability. We also discuss related work on issue categorization, which supports automated repair by identifying common patterns of degradation in code.

A notable example is given by Xia et al. (2023), which systematically investigates how large pre-trained language models perform on established program repair benchmarks. The study evaluates models such as Codex and InCoder on Defects4J version 1.2, a benchmark of real-world Java bugs including logical errors, API misuse, faulty exception handling, incorrect algorithms, and other functional defects mined from open-source projects. The evaluation spans three repair settings: full function generation, single line generation, and code infilling. The results demonstrate that LLMs can match or even outperform traditional automated program repair tools in the number of successfully repaired bugs, even without fine-tuning. For example, Codex repaired 40 percent of bugs in Defects4J (62 out of 154) using localized code infilling, compared to 28 percent (63 out of 225) when generating an entire function, showing the advantage of focused, context-aware repair. Likewise, InCoder, generating 2000 samples and guided by simple repair templates, fixed 78 bugs, including 15 unique cases not fixed by any baseline technique. These findings highlight that LLMs, especially when combined with lightweight domain-specific strategies, can effectively repair real semantic bugs in software, including logic and correctness defects typical of production code.

A focused effort on LLM-driven vulnerability repair is presented in Berabi et al. (2024), which explores how large language models can be guided to fix real-world security issues using precise contextual information. The approach introduces a targeted patching framework where only code relevant to a static analysis warning is passed to the model, improving both accuracy and safety. They evaluate results using Pass@k, which measures how often at least one of the top k generated fixes resolves the vulnerability without introducing new ones. For SecurityFlow issues, which involve complex interprocedural dataflows, their system achieved strong results, with top models reaching 81.62% Pass@5, specifically Mixtral-8x7B-CodeReduced.

The paper (Seo et al. 2025) presents another study focused on vulnerabilities in code generated by large language models, introducing AutoPatch, a multi-agent framework designed to detect and fix real-world security flaws that were disclosed after the models' training cutoff. AutoPatch compares generated code against a database of recent high-severity vulnerabilities from software like the Linux kernel and Chrome, using semantic and taint analysis to identify similar issues. Once a match is found, it verifies whether the code is vulnerable and then generates a secure patch through an iterative feedback loop. The system was evaluated on 525 code samples based on 75 recent vulnerabilities, achieving 89.5% accuracy in detecting issues and 95.0% accuracy in patching them, while being over 50 times more cost-effective than retraining-based methods.

An approach closely aligned with the objectives of automated static issue remediation is introduced in Wadhwa et al. (2024), which presents CORE, a system designed to resolve code quality issues using LLMs. CORE operates by leveraging two distinct models, one that proposes code edits (GPT-3.5-Turbo) based on static analysis diagnostics and another that evaluates and ranks those edits using a rubric informed by software engineering best practices (GPT-4). The system specifically targets issues reported by rule-based analyzers such as CodeQL, automatically filtering out suggestions that fail to satisfy the original rules. This dual-model architecture ensures not only syntactic compliance but also semantic plausibility, increasing the likelihood that proposed fixes align with developer expectations. In terms of static analysis outcomes, CORE achieved strong results on the Python benchmark (CQPy), successfully revising 88.81% of Python files such that they passed all relevant

CodeQL static checks. Furthermore, in a stricter setting requiring acceptance by both the static tool and human reviewers, CORE successfully revised 59.2% of files, showing its ability to produce fixes that meet both technical and human standards.

The research (Abtahi and Azim 2025) presents a two-step process involving GPT-3.5 Turbo and GPT-4o to fix code issues identified by SonarQube, including bugs, vulnerabilities, and code smells. The approach operates as follows: each file is first revised using GPT-3.5 Turbo, and any remaining issues are then addressed by GPT-4o. For bugs, GPT-3.5 Turbo resolved 50% of the issues, while GPT-4o handled the rest, resulting in 100% combined resolution. For vulnerabilities, GPT-3.5 Turbo fixed 96.7%, and GPT-4o resolved the remaining cases, again reaching 100%. Code smells proved more challenging: GPT-3.5 addressed 50.9%, GPT-4o resolved 61.8%, and together they achieved 81.2%. When all issue types were revised in a single prompt per file, the combined success rate was 71.6%, highlighting the benefit of using the models together, where GPT-4o corrected what GPT-3.5 missed. These results were obtained from a single large-scale software project, the "EIS" system developed by Team Eagle Inc., which included 7,599 code issues across JavaScript, Python, Java, and Docker files, making it a diverse and realistic testbed for automated code revision.

Not all studies in this space focus directly on automated issue repair, but several illustrate how LLMs can leverage static analysis outputs. For example, Jaoua et al. (2025) integrates a fine-tuned CodeLlama-7B with PMD and Checkstyle to enhance automated code reviews via three strategies: data-augmented training (DAT), retrieval-augmented generation (RAG), and naive concatenation (NCO). RAG delivers the greatest accuracy gains, while DAT achieves the highest coverage, with 49% of reviews rated excellent; RAG also performs strongly, with 70% in the top two quality tiers. Evaluated using a Llama-3 70B judge, these results highlight the broader potential of hybrid LLM–static analysis methods for contexts such as automated issue repair in SonarQube, where combining rule-based precision with generative flexibility could improve both detection and remediation of code issues.

Another line of work relevant to automated issue repair focuses not on patch generation but on understanding and classifying quality-related concerns in code. The study (Shivashankar et al. 2025) proposes BEACon-TD, an ensemble-based approach using fine-tuned transformer models to detect and categorize technical debt (TD) across 13 distinct types. Rather than generating patches, the system enables targeted remediation by identifying TD types such as architectural (poor modularization) and test debt (missing or inadequate test coverage). For example, BEACon-TD achieved an F1-score of 0.892 on the test data from GitHub issues, outperforming GPT-4o and demonstrating strong generalization on out-of-distribution datasets. These results suggest that high-quality classification can serve as a foundation for prioritizing repair actions and guiding automated maintenance workflows.

In contrast to prior work, which predominantly focuses on repairing bugs, vulnerabilities, or isolated static analysis findings using specialized frameworks or hybrid approaches, our study systematically evaluates the effectiveness of publicly available, general-purpose LLMs on a diverse set of real-world Python projects and a wide spectrum of SonarQube-reported issues, covering multiple kinds of problems rather than focusing narrowly on a single category such as security. Rather than proposing a novel repair algorithm or combining multiple models, we assess the ability of individual models to generate syntactically correct, maintainable, and rule-compliant fixes directly from standardized prompts. Our work dif-

fers by grounding the evaluation in a unified, reproducible framework that integrates model outputs into actual codebases and measures their impact through static analysis and syntactic validation, offering a comprehensive and practical benchmark of LLM performance in addressing SonarQube-detected issues.

4 Study Design

Our research was designed as an evaluation study, aiming to systematically assess the effectiveness of large language models (LLMs) in addressing code quality issues reported by SonarQube. Rather than proposing a novel repair algorithm, our goal is to critically examine how well existing, publicly accessible LLMs perform in interpreting static analysis output and generating plausible, rule-aligned code fixes. We conduct this evaluation in a controlled setting to ensure consistency and reproducibility of the assessment. This orientation aligns with foundational perspectives on software engineering evaluation, where the focus lies in the practical assessment of tools within concrete operational contexts (Wieringa et al. 2006).

In the Goal-Question-Metric approach (Basili et al. 1994), the objective of our study can be summarized in: Evaluate the performance of LLMs to repair detected code issues in the context of open source Python projects, which we further divide in the following research questions:

- **RQ1** How effective are LLMs in repairing **project**-level quality issues?
- **RQ2** How effective are LLMs in repairing **function**-level quality issues?
- **RQ3** How effective are LLMs in repairing **issue**-level quality issues?
- **RQ4** How effectively do LLMs preserve the **semantic behavior** of the original functions when generating repairs?

This study examines the extent to which current LLMs can interpret and act upon structured diagnostic information produced by static analysis tools. Static analysis tools offer precise and actionable feedback based on a well-defined taxonomy of rule violations. However, the question of whether LLMs can understand and respond appropriately to these rule violations, particularly reliably and consistently, remains open. Addressing this question through systematic evaluation enables us to examine the strengths and limitations of current LLMs when deployed for automated issue repair in production-like scenarios.

In this evaluation workflow, each LLM receives standardized prompts constructed from static issue metadata along with carefully selected contextual code snippets. The selected models are invoked under consistent prompting conditions to ensure a fair and reliable basis for comparison across different systems. Once the outputs are generated, they are processed and examined using automated techniques. These include checks for syntactic correctness, as well as structured assessments that examine how well the responses reflect the intended semantics and whether the proposed changes align with the nature of the input context. This evaluation setup is designed to capture not just surface-level precision but also the model's ability to produce coherent and meaningful code modifications grounded in the surrounding logic.

We frame our study within established software engineering experimentation standards, following guidelines from Solari et al. (2018) to ensure transparency, replicability, and trace-

ability through detailed documentation of conditions, prompts, and evaluation criteria. This framing also highlights how LLMs could complement static analysis tools by automatically addressing code issues. While LLMs offer a scalable mechanism for such automation, their reliability remains uncertain. Our evaluation clarifies when LLMs can act autonomously and when human oversight is still necessary, providing a foundation for integrating them into code review workflows.

4.1 Dataset

This study builds upon the dataset introduced in Moldovan et al. (2024), which was specifically constructed to collect static analysis results from SonarQube, refactoring data from PyRef, and defect history computed using the SZZ algorithm. That dataset was designed to support research on software quality and maintenance by providing a rich set of metrics and historical insights. Although SonarQube analyses for multiple historical releases of each project are already included in the dataset, we decided to take the latest version of each repository directly from GitHub and execute a fresh SonarQube analysis ourselves, following the methodology described in the original work. This decision ensures that our evaluation reflects the current state of the projects' codebases, avoids outdated or inconsistent findings from historical snapshots, and provides an updated view of the code quality aligned with their most recent development. By recomputing the SonarQube results on the latest code, we align our evaluation with the present-day reality of these projects while maintaining methodological consistency with prior studies.

Although we could have used other Python-specific tools, such as pylint and ruff, which are widely adopted in open-source Python projects, we decided to use SonarQube, as in the original dataset, in order to rely on a more general and standardized static analysis tool. SonarQube provides a mature Python ruleset covering code smells, bugs, and vulnerabilities, and supports consistent evaluation across projects.

The original dataset contains 80 open-source Python projects spanning a broad range of domains, development practices, and codebase sizes, providing a rich foundation for large-scale analysis of software quality and automated repair tools. For the detailed project-level study, we focused on 20 repositories that more closely reflect production-grade software, projects that are widely adopted, actively maintained, and representative of mature open-source development. Selecting this subset makes it easier to analyze each project individually while preserving diversity and real-world relevance. The choice was guided primarily by the **number of forks**, used as an indicator of popularity, reuse, and community engagement within the open-source ecosystem.

In GitHub, a **fork** is an independent copy of a repository created by a developer to modify or extend the original code. The number of forks reflects how many times a project has been duplicated for further experimentation or reuse, capturing both its visibility and the level of community engagement. The selected repositories are approximately in the top quartile of the dataset in terms of fork count, underscoring their status as some of the most widely adopted and actively reused open-source Python projects.

To ensure that the analysis relied on up-to-date information, the **GitHub API** was used to verify and retrieve the most recent data for each repository. This update provided accurate values for the number of forks and also included the total **number of commits**, offering additional insight into the current level of project activity and maintenance. These refreshed

metrics ensured that the selected repositories reflect their current state rather than outdated snapshots.

The observed commit activity demonstrates continuous maintenance and improvement, confirming that the selected repositories are not only popular but also technically mature and actively evolving. The combination of high fork counts and ongoing development ensures that the analyzed projects represent realistic, living examples of modern open-source software ecosystems. The final selection also preserves diversity across software domains, including machine learning (e.g., Scikit-learn, Keras), data analysis (e.g., Pandas, Matplotlib), web frameworks (e.g., Django, Django REST Framework), and enterprise or automation systems (e.g., ERPNext, Ansible). Table 1 presents the selected repositories together with their number of commits and their number of forks.

We executed SonarQube (version 10.0.0) using the default Python ruleset for all selected repositories. The analysis extracted the type of each issue (code smell, bug, or vulnerability), a brief description, and its precise location in the source code. These raw results form the basis of the per-project totals presented in Table 2.

Across all 20 repositories, SonarQube identified a total of 46,479 issues, consisting of 44,793 code smells, 1,649 bugs, and 37 vulnerabilities, as shown in Table 2. This distribution highlights the structural dominance of code smells in Python projects, a trend consistently reported in similar studies like Molnar and Motogna (2020). To provide a broader overview of the dataset, Table 3 summarizes descriptive statistics for each issue category, including minimum, maximum, average, and median values across projects.

While SonarQube reports the specific lines associated with each issue, such fragments do not provide sufficient context for a language model to understand the control flow or data dependencies of a function. For this reason, after identifying the issues summarized in Tables 2 and 3, we expanded each instance to include the full function in which the

Table 1 Repository statistics

Project	Commits	Forks
Scikit-learn	33098	26389
Saleor	22205	5843
Pandas	36671	19222
Django	34001	33180
Erpnext	54837	9705
Ansible	55218	24113
Matplotlib	53380	8104
Yt-dlp	23550	10734
Mmdetection (MMDet)	2706	9777
Fairseq	2330	6621
Zulip	67088	8856
Openpilot	16126	10375
Celery	12930	4881
Keras	11775	19646
Scrapy	10938	11140
Jumpserver	12683	5587
Stable diffusion webui (SD WebUI)	7689	29280
Django rest framework (Django RF)	8912	7029
Tornado	4930	5539
Requests	6379	9579

Table 2 Distribution of SonarQube issues by type

Project	SQ Issues	Code Smells	Bugs	Vulnerabilities
Scikit-learn	7731	7623	108	0
Saleor	5829	5744	85	0
Pandas	5789	4829	954	6
Django	4329	4241	87	1
Erpnext	3847	3808	38	1
Ansible	2808	2762	42	4
Matplotlib	2350	2322	28	0
Yt-dlp	1890	1862	28	0
Mmdetection	1756	1715	41	0
Fairseq	1574	1538	36	0
Zulip	1468	1463	4	1
Openpilot	1416	1406	10	0
Celery	1237	1213	23	1
Keras	1161	1139	22	0
Scrapy	958	888	61	9
Jumpserver	628	589	35	4
Stable-diffusion-webui	562	551	11	0
Django-rest-framework	531	517	14	0
Tornado	508	476	22	10
Requests	107	107	0	0

Table 3 Overall statistics for SonarQube issues by type

Issue Type	Minimum	Maximum	Average	Median
Total Issues	107	7731	2323.94	1521.00
Code Smells	107	7623	2239.65	1500.50
Bugs	0	954	82.45	31.50
Vulnerabilities	0	10	1.85	0.00

issue appears. This ensured that every example used in the repair tasks contains complete structural and semantic context, enabling the models to reason about variable scope, logical structure, and surrounding behavior.

While expanding the context to the full module could, in theory, provide additional information relevant to certain issues, some modules in the analyzed projects are quite large, often spanning hundreds or even thousands of lines. Providing such extensive input to the models tends to dilute relevant information and raise the likelihood of incomplete or truncated responses. The function level scope, therefore, represents a practical balance, as it provides sufficient local context for most issue types while keeping the prompt size manageable and the inference process consistent across all evaluated models.

To extract the complete function associated with each SonarQube-reported issue, we implemented a Python script that operates based on syntactic cues. Starting from the line number indicated in the SonarQube report, the script traverses upward through the source file until it locates the nearest enclosing **def** statement, which marks the beginning of the function. From that point, it moves downward line by line, collecting all subsequent lines that maintain the same indentation level or greater. This indentation-based traversal enables

the script to determine the full body of the function, stopping once it encounters a line that is no longer part of the same block or logical scope.

This approach takes advantage of Python's reliance on indentation to define code blocks, making it well-suited for reliably identifying function boundaries. The script also incorporates handling for common edge cases, such as the presence of inline comments or blank lines within the function body. Decorators that appear immediately above a function are not included in the extracted code segment, as the extraction process begins at the corresponding `def` statement. Internal docstrings, internal comments, and internal blank lines are preserved to maintain the original structure and logical flow of the code, while leading and trailing blank lines are trimmed to ensure clean boundaries. By extracting entire functions in this manner, we ensure that language models receive a coherent and context-rich input, allowing them to reason about control structures, variable lifecycles, and semantic dependencies, all of which are critical for generating valid and meaningful repairs.

To facilitate structured evaluation and support reproducibility, we grouped the extracted issues by their corresponding functions. Each function was treated as a self-contained unit, paired with one or more SonarQube-reported issues. This mirrors how developers typically resolve related problems in a single editing session, and it ensures that models have access to all diagnostics relevant to the same local context. Grouping also avoids conflicting or incomplete fixes that might arise if issues were processed in isolation. Crucially, it enables a holistic evaluation of the model's ability to understand and address multiple interacting concerns within the same logical scope.

As shown in Fig. 1, multiple issues can co-occur within a single function and often overlap in scope or resolution. In this example, the function contains warnings about cognitive complexity, an unused parameter, a naming convention violation, and a regular expression style issue. Grouping them enables the model to reason about their relationships and propose coordinated changes. This function-level organization provides a realistic input granularity for LLMs and sets the foundation for evaluating more complex scenarios, such as multi-issue resolution or repair quality under varying issue densities.

This designed dataset and extraction process lay the groundwork for a robust and realistic evaluation of LLM-assisted code repair. By selecting mature and diverse projects, recom-

Fig. 1 Example of multiple SonarQube issues reported within one function

puting up-to-date static analysis results, and providing complete, context-rich function-level inputs, we ensure that the evaluation reflects practical challenges faced by developers.

4.2 Selection of Large Language Models

In our work, we treat SonarQube issue reports as entry points for initiating LLM-driven code repair. By extracting rule identifiers, severity levels, diagnostic messages, and the corresponding code segments, we construct prompts that emulate realistic developer scenarios: receiving a structured warning and crafting an appropriate patch. This setup allows for a focused evaluation of each model's ability to understand diagnostic signals, reason over localized context, and generate improvements that preserve functionality while enhancing code quality. Our approach is designed to bridge academic knowledge on LLM-based program repair with real-world, tool-assisted development workflows.

We focused on general-purpose LLMs because recommendations from modern coding agents and IDE-level assistants indicate that most models used in practice are general-purpose systems. Both OpenCode and Cursor predominantly recommend models such as Claude Opus and Sonnet, and Gemini Pro, which combine code generation with program understanding and natural language interaction.[23] Moreover, addressing SonarQube-reported issues requires conceptual understanding of rule descriptions and diagnostics, making it primarily a problem-solving task rather than a simple code completion task.

A growing body of evidence supports the practical utility of LLMs in software quality improvement. For example, Simões and Venson (2024) demonstrated that GPT 3.5 and GPT-4o can assess maintainability while considering factors like naming conventions and readability often missed by traditional tools. Building on such findings, we selected six LLMs for their strengths in technical debt remediation, namely GPT-4o (gpt-4o-2024-08-06), Claude 3 Opus (claude-3-opus-20240229), Gemini 2.0 Flash (gemini-2.0-flash-001), DeepSeek V3 (deepseek-v3-0324), Grok 3 (grok-3), and Mistral Large (mistral-large-2411).

Table 4 lists the six models in this study, their producers, and official documentation links. These models span diverse architectures and have shown strengths in vulnerability detection, refactoring, standards-compliant code generation, code smell identification, and runtime error repair. Our goal is to assess how well each interprets SonarQube rule-based diagnostics and produces maintainable, context-aware code changes that reduce technical debt.

The inclusion of GPT-4o in our study is motivated by its demonstrated effectiveness in technical debt detection and mitigation tasks, particularly in the domain of build systems. A recent study (Ghammam et al. 2025) evaluated its use as part of the BuildRefMiner tool, which automatically identifies and classifies refactorings in build scripts. In that study, GPT-4o was used to classify each refactoring from a dataset into one of 24 classes (e.g., Rename Field, Remove Unused Code) using prompts with class-specific examples. With this prompt-based approach, the model achieved 79% precision and 78% recall, accurately detecting a range of refactoring patterns and attaining performance suitable for supporting automated build maintenance. These results suggest that GPT-4o can handle diverse refactoring categories with balanced accuracy, making it a relevant candidate for code quality improvement tasks.

[2] https://opencode.ai/docs/models/

[3] https://cursor.sh/docs/models

Table 4 Large language models, producer, and official links

Model	Producer	Official Link
GPT-4o	OpenAI	https://openai.com/research/gpt-4o
Claude 3 Opus	Anthropic	https://www.anthropic.com/news/claude-3-family
Gemini 2.0 Flash	DeepMind	https://deepmind.google/technologies/gemini/
DeepSeek V3	DeepSeek	https://deepseek.com/
Grok-3	xAI	https://x.ai/
Mistral Large	Mistral AI	https://mistral.ai/

We selected Claude 3 Opus for this study because of its performance in software vulnerability detection tasks, which align closely with the types of security-related technical debt identified by SonarQube. In a comprehensive benchmarking study (Tamberg and Bahsi 2025), Claude 3 Opus was evaluated against multiple large language models for its ability to identify software vulnerabilities across seventeen categories defined by the Common Weakness Enumeration (CWE), a standardized taxonomy of software security weaknesses. The model outperformed traditional static code analysis tools such as CodeQL and SpotBugs, demonstrated the strongest ability among the evaluated LLMs to generalize across vulnerability types, missing vulnerabilities in only 3 of the 17 categories, and achieved the lowest false positive rate at 11.6%.

We included Gemini 2.0 Flash in our study because of its demonstrated capability to understand and reason about source code, which is essential for identifying and fixing software issues. As shown in prior evaluations (Cihan et al. 2025), Gemini 2.0 Flash achieved a correctness classification accuracy of 63.89% and a correction ratio of 54.26% on a mixed dataset of 492 AI-generated Python code blocks, with performance improving when provided with problem descriptions. This ability to comprehend code logic makes it a promising candidate for addressing issues in SonarQube that stem from cognitive errors, where the fix requires not just syntactic adjustments but also an accurate mental model of the intended functionality. Such a deeper understanding could provide a distinct advantage in improving the quality of automated issue resolution.

DeepSeek V3 was selected for its capability in detecting code smells, contributing to improved code quality and maintainability. The study (Zhang et al. 2024) compared DeepSeek V3 with GPT-4o-mini across four common code smell types: Complex Conditional, Complex Method, Feature Envy, and Data Class. DeepSeek V3 generally achieved higher precision, indicating greater accuracy in identifying true positives without overflagging clean code. The model achieved 72.38% compared with 71.48% for Complex Conditional and 79.74% compared with 79.06% for Complex Method. GPT-4o-mini attained a slight advantage for Feature Envy, at 77.53% compared with 74.29%, whereas DeepSeek V3 showed a clear lead for Data Class, at 87.57% compared with 84.87%, making it the more precise tool in most categories.

In Bucaioni et al. (2025), Grok Beta, an earlier version of xAI's Grok series, achieved a 69.74% success rate in repairing runtime errors in C++ and Java. This outperformed lower-ranked models such as Llama 3.2, which scored 30.26%, and came close to GPT-4o's 72.36%, one of the top performers in the evaluation. The small gap between Grok Beta and GPT-4o indicates that even the earlier Grok was competitive with the best available models. Our study uses Grok 3, the direct successor to Grok Beta, incorporating architectural

and training enhancements, and we expect it to match or exceed GPT-4o's performance in autonomous runtime error repair.

Mistral Large demonstrates a structured reasoning ability that can identify patterns of coding issues, suggesting it could be a suitable candidate for inclusion in this study. In a vulnerability detection benchmark presented in Gnieciak and Szandala (2025), it achieved a precision of 78.3% and a recall of 78.3%, showing competitive results with DeepSeek V3, which scored 72.3% precision and 84.7% recall. While DeepSeek V3 tended to detect more vulnerabilities, this came with a higher likelihood of false positives. These results indicate that Mistral Large offers a balanced performance profile, which may be advantageous when aiming to improve detection without substantially increasing verification effort.

We selected six models to combine architectural diversity with domain-specific strengths relevant to technical debt mitigation and static analysis repair. GPT-4o and Claude 3 Opus provide broad reasoning capabilities applicable to a range of defect categories, with particular strengths in refactoring classification and vulnerability identification. DeepSeek V3 and Grok 3 contribute capabilities in code quality assessment, including code smell detection and runtime fault repair. Gemini 2.0 Flash offers strong code comprehension for addressing cognitive and logic-related issues, while Mistral Large provides balanced detection performance across precision and recall metrics. Table 5 synthesizes the principal selection criteria for each model.

4.3 LLM-Based Code Repair: Execution Workflow and Validation

This subsection describes the workflow we developed to generate, apply, and validate code repairs using large language models (LLMs) on SonarQube-reported issues. The process was designed to be systematic, reproducible, and comparable across models, while addressing practical challenges such as syntax validation and imperfect function extraction.

We started from our described dataset with groups of functions and their associated SonarQube-reported issues, which had been extracted from the original projects in advance. In order to evaluate each model separately and fairly, we prepared six copies of each project, one for each LLM, ensuring that the repairs generated by each model were applied only to its corresponding copy. Each repair was generated independently from the original, unmodified function, ensuring that LLMs never saw previously-generated fixes during the generation phase. The generated fixes were then applied sequentially to their respective project

Table 5 Primary selection criteria for LLMs in this study

Model	Primary Selection Criterion
GPT-4o	Performance in refactoring classification and build maintenance
Claude 3 Opus	Generalization across vulnerability categories with low false positives
Gemini 2.0 Flash	Accurate comprehension of program logic and cognitive issues
DeepSeek V3	Precision in identifying code smells across smell types
Grok 3	Effectiveness in automated repair of runtime faults
Mistral Large	Balanced precision–recall in vulnerability detection

copies, with each transformation representing an independent remediation attempt based solely on the original code and its associated SonarQube issues.

For each function in the dataset together with its associated group of SonarQube-reported issues, the fixing process was carried out in three main steps. **The first step** was to generate a repair for the function. We constructed a prompt containing a natural language summary of the SonarQube-reported issues, the full source code of the function, and an instruction to produce a corrected version. The summaries were directly generated from SonarQube outputs by extracting, for each finding, the rule identifier, diagnostic message, and the code snippet from the line of code where the issue started, as reported by SonarQube. Each prompt was sent independently to the API of each model, resulting in one refactored version of the function per LLM, each returned inside a code block.

To ensure consistency and comparability across models, all prompts followed a fixed structure and used the same content. The instruction emphasized preserving functionality, improving readability, and following Python best practices. Models were explicitly instructed to return only the corrected function, enclosed in a code block, to enable automated post-processing. Listing 1 shows an example of the prompt template used.

This standardized procedure ensured a fair and reproducible evaluation by holding inputs constant across all models, eliminating variability caused by prompt differences. By using SonarQube summaries and a fixed template, we guaranteed that each model processed the same information under identical conditions, enabling a direct and reliable comparison of their code-repair capabilities while supporting seamless automated validation.

```
Please help fix the following code issues:

python:S1481 - Remove the unused local variable "result".
Code: result = get_file_content(self.FILE, default='', strip=True)

python:S112 - Replace this generic exception class with a more specific
↪ one.
Code: raise Exception("Failed to read hostname file")

Here's the code to fix:

def get_permanent_hostname(self):
    if not os.path.isfile(self.FILE):
        return ''
    try:
        result = get_file_content(self.FILE, default='', strip=True)
        return result
    except IOError as e:
        raise Exception("Failed to read hostname file")

Please provide a refactored version that addresses these issues.
Focus on:
1. Maintaining the same functionality
2. Keeping the code readable and maintainable
3. Following Python best practices

Return only the fixed code without any explanations.
Wrap the code in ```python...```.
```

Listing 1 Prompt with SonarQube summary, function code, and repair instruction

Several alternative prompting strategies were considered during the design phase, but were deliberately not adopted. Retrieval-augmented prompting (RAG) was excluded because repair quality would depend on the embedding model and retrieval setup, introducing additional variability beyond the LLMs themselves. We also avoided a pure repair setting without explicit issue descriptions, as this would shift the task toward issue detection rather than issue remediation. Few-shot and chain-of-thought prompting were likewise not employed, as example selection and intermediate reasoning instructions can bias model behavior and reduce reproducibility in a multi-model comparison. We therefore adopted a single zero-shot prompt that provides SonarQube diagnostics and function-level context, ensuring a fair and controlled evaluation of repair capabilities.

Claude 3 Opus and Gemini 2.0 Flash were configured to respond using their default temperature values. Prior empirical work (Renze 2024) reports that varying temperature between 0 and 1 yields minimal performance differences across diverse tasks and model families, including GPT models, Claude 3 Opus, and Gemini. In contrast, research focused specifically on code generation (Arora et al. 2024) shows that lower temperatures can improve correctness for GPT-based models by reducing sampling entropy. Since the task in this study centers on structured code repair rather than open-ended generation, GPT-4o was configured with a temperature of 0.7 to allow meaningful structural variation while maintaining stable and grounded output.

DeepSeek V3, Mistral Large, and Grok 3 were configured with a temperature of 0.1. This setting aligns with provider guidance that encourages low-temperature sampling for programming tasks. DeepSeek's documentation recommends a temperature of 0.0 for code-related tasks.[4] Mistral's documentation similarly suggests low-temperature sampling for code-focused use cases.[5] Evaluations released by the model provider indicate the use of temperatures around 0.1 for programming-oriented tasks, and the same configuration was applied to Grok 3.[6] Using a shared low temperature for these models reflects established conventions for deterministic code transformation while still allowing minimal controlled variation.

The second step was to replace the original function in the appropriate project copy with the generated refactored version. For each model, its corresponding refactored function was inserted into that model's dedicated copy of the project repository. A Python script was used to locate the original function in the file, replace it with the new version while preserving correct indentation, and save the updated file. This script operated directly on the functions and the groups of issues associated with them, as specified in the dataset, ensuring that the repairs were applied exactly where the issues had been detected. The script worked by searching for the specific function definition in the code, matching it based on its indentation and structure as recorded in the dataset.

While this method worked reliably in most cases, it was not without limitations. In some situations, the dataset contained incorrectly parsed functions, which made them impossible to locate in the project files. Other failures arose when functions included nested definitions, such as helper functions defined inside another function, or when annotation lines and comments were shared between adjacent functions in a way that confused the matching logic.

[4] https://api-docs.deepseek.com/quick_start/parameter_settings

[5] https://docs.mistral.ai/capabilities/code_generation

[6] https://x.ai/news/grok

These edge cases occasionally prevented the replacement from succeeding. Nonetheless, the process provided a systematic and automated way to apply the fixes across the projects.

The third step was to validate the updated code. After replacement, we used Python's **abstract syntax tree (AST)** module to verify the parsability of the modified file. The AST module parses Python source code and builds an abstract syntax tree that represents the structural organization of the program. This step ensures that the generated code conforms to Python grammar rules and can be successfully parsed. If the AST check failed, meaning that the code could not be parsed and a syntax error was raised, the change was reverted, and the original function was restored. This validation was necessary to ensure that only structurally valid updates were retained in the project copy and to remain consistent with static analysis workflows, such as SonarQube, which rely on AST parsing as a prerequisite for rule evaluation.

Figure 2 summarizes the entire repair process of a function using a flowchart. After prompt construction and model inference, the generated code is integrated and validated. If the AST validation succeeds, the change is retained; otherwise, the function is reverted to its original form.

After all the functions in each project were processed through the described steps of refactoring, replacement, and validation, we obtained six updated versions of each project, each corresponding to the output of one of the six LLMs. At this stage, for each of the twenty projects, there was a distinct repository version for each model. Once these updated versions were prepared, we executed SonarQube again on each of them to evaluate the impact of the repairs. We measured how many of the previously reported issues were resolved in each version, as well as how many issues remained or were added. This final step provided a systematic and consistent way to assess the effectiveness of the LLMs in addressing SonarQube-reported issues across a diverse set of projects.

This end-to-end workflow, beginning with prompt construction and model inference, continuing through function replacement and AST validation, and concluding with SonarQube re-analysis, provided a robust, realistic, and reproducible framework for evaluating LLM-assisted code repair at scale. By combining standardized inputs, controlled experimental environments, and rigorous validation steps, the methodology ensured that the results accurately reflected both the strengths and the limitations of current LLMs in addressing real-world static analysis feedback. This comprehensive process not only allowed for consistent comparison across different models but also highlighted practical challenges and opportunities for integrating LLMs into automated code maintenance workflows.

To illustrate the types of repairs generated by the models, we present an example of a function before and after LLM-based refactoring, as shown in Figs. 3 and 4. The original function, responsible for handling the resolution state of an order, exhibits a high degree of cognitive complexity due to deeply nested conditional logic. Such nesting impairs readability, increases maintenance difficulty, and raises the risk of logical errors. Additionally, the function declares a parameter, **trace_context**, which is never used, a common code smell flagged by static analyzers. Together, these issues exemplify the structural and clarity-related concerns that LLMs were tasked with addressing during the repair process.

In the refactored version, generated by one of the LLMs, the nested conditionals are flattened and the unused parameter is removed, as shown in Fig. 4. These improvements make the code more readable, maintainable, and aligned with Python best practices, demonstrating the model's ability to effectively address structural problems. However, it is important to

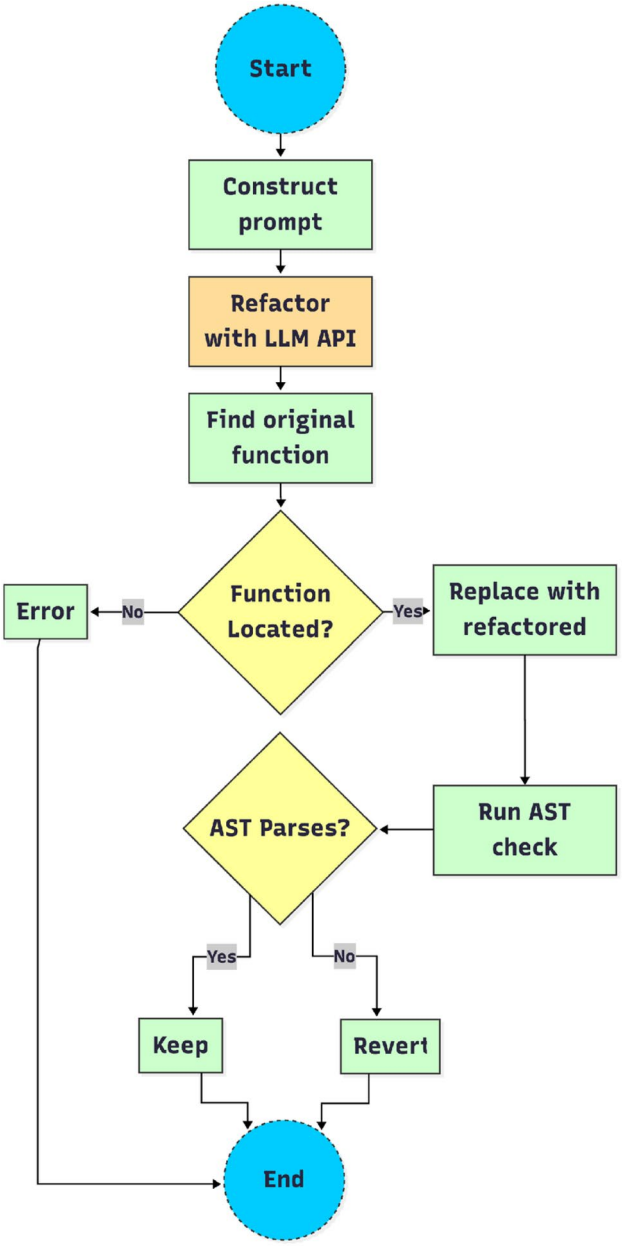

Fig. 2 LLM-Based issue repair workflow

Fig. 3 Example of a function before LLM refactoring

Fig. 4 Example of the same function after LLM-generated refactoring

note that while the repair reduces complexity and improves code hygiene, the function still retains some inherent complexity due to the business logic it implements. This underscores both the potential and the limitations of LLM-based repairs, they can address specific, well-scoped issues such as redundant parameters and excessive nesting, but achieving a fully optimal design may still require further human review and broader refactoring beyond the scope of what was requested in the prompt.

In some cases, such as functions with high cyclomatic complexity, the appropriate solution involved extracting helper methods or reorganizing logic to reduce complexity. As mentioned earlier, these cases were treated impartially, without any special handling or prompt adjustments. Each LLM received the same standardized input and independently generated its refactored version. The resulting code, whether it involved function extraction or other structural changes, was directly used to replace the original function and validated through the same automated process.

The presented workflow offers a systematic and reproducible approach to evaluating LLM-assisted code repair, combining prompt-based generation, automated integration, and rigorous validation. By applying this process consistently across 20 diverse projects and 6 different models, we established a solid foundation for assessing their ability to remediate real-world SonarQube issues.

4.4 Test-Suite-Based Validation Protocol

To complement static-analysis-based evaluation and address concerns regarding behavioral correctness, we designed a test-suite-based validation protocol that assesses whether function-level repairs generated by large language models preserve the expected runtime behavior of the original code. This protocol is part of the study design and provides execution-based evidence of correctness, independent of static diagnostics or LLM-based semantic judgments.

For each of the selected projects, we constructed a dedicated Docker environment to ensure reproducibility and isolation of dependencies. Each Docker image encapsulates the project-specific Python version, required libraries, and testing framework, and executes either the standard `pytest` command or a project-specific test script. All test executions were performed within these controlled environments.

As a baseline, we executed the test suite of each project on the original, unmodified codebase. For every execution, we recorded a summary of the test outcomes, including the aggregated numbers of passed, failed, errored, skipped, xfailed, and xpassed tests. This summary was extracted from the final lines of the test execution logs, where the testing framework reports the overall results of the run. These baseline summaries serve as a reference for all subsequent comparisons and allow us to attribute observed behavioral changes to LLM-generated code modifications rather than to pre-existing test failures.

To enable causal attribution between individual repairs and test outcomes, we adopted an iterative single-change validation strategy. Exactly one function-level modification was applied to the codebase at a time, after which the test suite was executed, and the corresponding execution logs were collected. The resulting test outcome summary was then compared against the baseline summary. After each execution, the codebase was reverted to its original state before proceeding to the next function. This setup ensures that any observed behavioral differences can be unambiguously associated with the specific repair under evaluation.

Because executing this procedure exhaustively for all modified functions would be computationally prohibitive, we employed a statistically informed sampling strategy. We selected a confidence level of 90% and a margin of error of 7% at the project level. These parameters represent a compromise between precision and feasibility, given that each evaluation requires a full execution of the project's test suite, which can be computationally expensive. Using stricter thresholds, such as 95% confidence or a 5% margin of error, would have substantially increased the number of required executions without proportionally improving the interpretability of the results in this exploratory setting. Under these assumptions, the resulting upper bound was 139 function-level repairs per project.

For projects containing fewer than 139 modified functions, all functions were evaluated. For projects exceeding this threshold, a random sample of 139 original functions was selected to ensure a manageable and comparable evaluation scope across models. The same

sampled set was then evaluated across all six LLMs. For each sampled function, the test suite was executed once per model, resulting in six independent executions per function, each applying the corresponding LLM-generated modification in isolation and reverting the codebase to its original state afterward. While sampling guarantees are defined per project, the study reports results across multiple independent projects, which provides a broader empirical perspective beyond any single codebase. The experiment was further complemented by an LLM-based semantic estimation using GPT-4o to assess whether the generated modifications preserve the original functionality of the code.

5 Analysis and Evaluation

5.1 Project-Level Repair Effectiveness

In this section, we provide an answer to RQ1 and we present a straightforward view of how six large language models (LLMs), Claude, DeepSeek, Gemini, GPT-4o, Grok, and Mistral, performed in addressing SonarQube-reported issues across 20 Python projects. Each model generated code repairs based on issues in the original source code. As we mentioned before, after applying the suggested fixes, the code was reanalyzed using SonarQube to assess how many issues remained. Table 6 presents the total number of Python issues before and after model intervention, sorted by original issue count to highlight trends across projects of different sizes (bold values indicate the best observed result in each row). This project-level view has limitations as it does not account for issue types, fix quality, or integration failures. Nonetheless, it provides a clear summary of the LLMs' raw impact and a useful baseline for deeper analysis.

As shown in Table 6, incorporating LLMs into the static analysis remediation process led to substantial reductions in SonarQube-reported issues. On average, the LLM-generated project versions achieved a 36.02% reduction in reported issues. These improvements were observed across Python projects of different sizes, structures, complexities, and application domains. Despite these differences, the LLMs consistently reduced issues, demonstrating their ability to generalize within the Python ecosystem. This indicates that LLM-assisted remediation is adaptable and effective at addressing diverse static analysis problems.

In Table 6, we also see the original number of issues for each project, as well as the new number of issues after applying fixes from each model, along with the percentage reduction achieved. For each project, the highest reduction achieved among all models is highlighted in bold in the table. Notably, Grok achieved the best reduction in 14 out of 20 projects, consistently outperforming the others in most cases.

To better understand and compare model performance, we complement the raw issue counts with a statistical analysis of the reduction percentages across all evaluated projects. While the previous table captures the absolute number of issues before and after model intervention, this additional analysis focuses on relative improvements and consistency. Such a perspective is essential for assessing the effectiveness of each model across heterogeneous project contexts and for identifying models that perform reliably across varying conditions. This enables more nuanced comparisons between models, supports evidence-based selection of models for deployment, and informs future optimization strategies.

Table 6 Number of remaining SonarQube issues per project for each model after LLM-based repairs, with reduction percentages from the original

Project	Initial	Claude	DeepSeek	GPT-4o	Gemini	Grok	Mistral
Scikit-learn	7731	3388	2873	2617	4484	**2616**	3211
		-56.18%	-62.84%	-66.15%	-42.00%	**-66.16%**	-58.47%
Saleor	5829	4771	**4762**	4777	4948	4800	4790
		-18.15%	**-18.31%**	-18.05%	-15.11%	-17.65%	-17.82%
Pandas	5789	2666	2696	2426	3115	**2357**	2978
		-53.95%	-53.43%	-58.09%	-46.19%	**-59.28%**	-48.56%
Django	4329	3081	2991	3122	3619	**2868**	3204
		-28.83%	-30.91%	-27.88%	-16.40%	**-33.75%**	-25.99%
Erpnext	3847	2962	2748	2753	3031	**2535**	3015
		-23.01%	-28.57%	-28.44%	-21.21%	**-34.10%**	-21.62%
Ansible	2808	1735	1647	1708	2269	**1620**	1836
		-38.21%	-41.35%	-39.17%	-19.19%	**-42.31%**	-34.62%
Matplotlib	2350	1288	1237	1142	1569	**1058**	1372
		-45.19%	-47.36%	-51.40%	-33.23%	**-54.98%**	-41.62%
Yt-dlp	1890	1462	1485	1453	1786	**1353**	1557
		-22.65%	-21.43%	-23.12%	-5.50%	**-28.41%**	-17.62%
MMDet	1756	1127	1004	919	1301	**840**	1159
		-35.82%	-42.82%	-47.67%	-25.91%	**-52.16%**	-33.99%
Fairseq	1574	997	1003	1046	1268	**996**	1062
		-36.66%	-36.28%	-33.54%	-19.44%	**-36.72%**	-32.53%
Zulip	1468	1211	1163	1123	1264	**1067**	1234
		-17.51%	-20.78%	-23.50%	-13.90%	**-27.32%**	-15.94%
Openpilot	1416	713	706	556	909	**403**	723
		-49.65%	-50.14%	-60.73%	-35.81%	**-71.54%**	-48.94%
Celery	1237	667	**646**	666	937	658	681
		-46.08%	**-47.78%**	-46.16%	-24.25%	-46.81%	-44.95%
Keras	1161	676	679	662	852	**601**	734
		-41.77%	-41.52%	-42.98%	-26.61%	**-48.24%**	-36.78%
Scrapy	958	708	**704**	715	910	711	741
		-26.10%	**-26.51%**	-25.37%	-5.01%	-25.78%	-22.65%
Jumpserver	628	369	345	345	423	**339**	385
		-41.24%	-45.06%	-45.06%	-32.64%	**-46.02%**	-38.69%
SD WebUI	562	415	400	419	462	**336**	412
		-26.16%	-28.83%	-25.44%	-17.79%	**-40.21%**	-26.69%
Django RF	531	338	333	328	406	**318**	339
		-36.35%	-37.29%	-38.23%	-23.54%	**-40.11%**	-36.16%
Tornado	508	299	303	**291**	429	296	327
		-41.14%	-40.35%	**-42.72%**	-15.55%	-41.73%	-35.63%
Requests	107	**51**	54	53	76	55	52
		-52.34%	-49.53%	-50.47%	-28.97%	-48.60%	-51.40%

In Table 7 a statistical summary of the performance for each model is provided. The columns include: the mean percentage of issue reduction (Mean), the standard deviation (Std Dev) indicating consistency, the minimum observed reduction (Min), the 25th percentile (25%), the median (Median), the 75th percentile (75%), and the maximum observed reduction (Max). This statistical evaluation approach is similar to the study presented in De Carv-

alho et al. (2021), which applies machine learning techniques to software effort estimation. Both approaches emphasize the importance of empirical metrics and variability analysis to assess predictive or transformative performance across heterogeneous software datasets.

To evaluate and compare the effectiveness of large language models in automated code remediation, we analyze both their average issue reduction and their standard deviation, which reflects the consistency of their performance. Standard deviation is a statistical measure that shows how much a model's results vary across different projects. A low standard deviation means the model performs consistently well, with relatively similar reductions across datasets, whereas a high standard deviation suggests uneven results, where some projects may benefit significantly while others experience only minor improvements. Considering both the mean and the spread of performance allows us to assess not only how effective each model is in general but also how dependable it is in diverse real-world scenarios, where predictability often matters just as much as peak performance.

To facilitate a structured comparison of model performance, we rank the evaluated LLMs into performance tiers using the Friedman test across the 20 projects. The Friedman test is a non-parametric statistical method commonly used to compare multiple algorithms across multiple blocks without assuming normality of the underlying distributions. In this setting, each project is treated as a block, and models are ranked within each project based on the number of remaining SonarQube issues after repair, with rank 1 assigned to the model that produces the fewest remaining issues. In this project-level view, applying the Friedman test is appropriate because it enables the comparison of model performance across heterogeneous codebases.

The resulting average ranks, reported in Table 8, are identical to the ordering obtained from the descriptive statistics in Table 7. In particular, the models follow the same ranking when considering the mean and median reductions, indicating that the observed performance differences represent stable trends across projects rather than being driven by isolated extreme cases.

Among the six evaluated models, Grok achieved the highest overall reductions in SonarQube-reported issues according to the Friedman ranking. It attained a mean reduction of 43.09%, with a maximum reduction of 71.54%. Its 25th, median, and 75th percentile values are consistently higher than those of the other models, indicating that it delivers substantial improvements across diverse project contexts. While its standard deviation (13.66) indicates some variability, this variation is consistent with its ability to produce the most substantial reductions observed in the evaluation. Following Grok, GPT-4o demonstrated similarly strong performance, with a mean reduction of 39.71% and a maximum of 66.15%, though with slightly higher variability (standard deviation 13.78). Together, these two models achieved the highest reductions in the study.

Table 7 Summary of issue reduction performance across LLMs by percentage

Model	Mean	Std Dev	Min	25%	Median	75%	Max
Grok	43.09	13.66	17.65	34.01	42.02	49.49	71.54
GPT-4o	39.71	13.78	18.05	27.27	40.95	48.37	66.15
DeepSeek	38.55	11.99	18.31	28.77	40.85	47.47	62.84
Claude	36.85	11.92	17.51	26.15	37.44	45.41	56.18
Mistral	34.53	12.15	15.94	25.16	35.13	42.45	58.47
Gemini	23.41	10.88	5.01	16.19	22.38	29.89	46.19

The next tier of models includes DeepSeek, Claude, and Mistral, which exhibit solid and competitive performance, albeit somewhat lower than the top two models. DeepSeek achieves a mean reduction of 38.55%, with a reasonably low standard deviation of 11.99, suggesting that it is both effective and fairly consistent across projects. Claude follows closely, with a mean reduction of 36.85% and a standard deviation of 11.92, reflecting reliable performance that remains competitive within this tier of models. Mistral rounds out this group, with a mean reduction of 34.53% and a standard deviation similar to that of Claude, reflecting comparable consistency. Although these models do not reach the peak reductions demonstrated by Grok or GPT-4o, their results remain competitive and relatively close, making them viable options, particularly in contexts where consistency, predictability, and robust performance across diverse projects are valued alongside effectiveness.

Finally, Gemini consistently trails the other models in this evaluation. It achieved a mean reduction of only 23.41%, with a minimum observed reduction as low as 5.01% and a maximum of 46.19%, both significantly below the best-in-class results. Additionally, its standard deviation of 10.88 indicates that it is not just less effective overall but also less variable, suggesting it consistently provides modest improvements regardless of project characteristics. While it may still contribute to some reduction in issues, its performance falls short of the other models and would likely not be the first choice when substantial remediation is desired. Exploring more advanced prompt engineering strategies could help unlock better performance from Gemini in similar scenarios.

This subsection shows that large language models (LLMs) have the capacity to address SonarQube-reported issues in Python projects effectively. The results indicate that LLMs can substantially reduce issue counts across diverse codebases, demonstrating their potential as practical tools for automated code remediation. However, this project-level view is limited, as it considers only aggregate issue counts without examining the specific fixes or their broader impact. It also does not account for cases where generated fixes introduce new issues or fail to integrate cleanly into the codebase. These aspects will be analyzed in detail in the following subsections.

5.2 Function-Level Repair Effectiveness

Now we will discuss the function-level performance of the evaluated models and provide a response to RQ2. While project-level results showed overall reductions in SonarQube-reported issues, they did not reveal how well models performed where code changes were actually applied. At this level, each issue was linked to its enclosing function, as we mentioned in Section 4.1. For each original function with reported issues, six corresponding refactored versions generated by the six LLMs were created, and each of these was inserted into the codebase and processed individually.

Table 8 Average ranks of LLM models according to the Friedman test across 20 projects (lower rank indicates better performance)

Model	Rank	Avg. Rank
Grok	1	1.60
GPT-4o	2	2.675
DeepSeek	3	2.725
Claude	4	3.35
Mistral	5	4.65
Gemini	6	6.00

Before the function-level evaluation, the initial dataset contained 46,479 Python issues, which were processed using our grouping algorithm, resulting in 30,702 candidate functions. Based on the extraction and replacement pipeline logs, the dataset was then cleaned by successively removing 1,715 entries mapped to classes rather than standalone functions, 709 entries consisting exclusively of non-functional elements such as imports or comments, 771 truncated functions, and 5,187 functions that could not be consistently located in at least one of the evaluated models. As a result of these dataset- and pipeline-level limitations, and independent of model behavior, the evaluation set was reduced from 30,702 to 22,320 valid functions, corresponding to the exclusion of 27.3% of candidate functions. This step ensured that all models were evaluated only on functions for which the pipeline was guaranteed to work.

After obtaining this set of 22,320 functions, we performed an additional structural validation pass to further ensure input integrity prior to evaluation. In this step, we applied lightweight syntactic sanity checks, including verifying that each code fragment began with a valid `def` statement and that parentheses were balanced throughout the function body. These checks were designed to detect rare residual extraction artifacts that were not identified during the earlier filtering stages. As a result, 61 additional functions exhibiting structural inconsistencies were excluded, yielding a final evaluation set of 22,259 functions.

The evaluation then proceeded on the remaining 22,259 functions. For this filtered subset, we examined parse errors detected during AST-based validation, marking a refactoring as a parsability failure when the model-generated function was inserted but failed AST-based parsing. Across the evaluated models, three performance tiers became clear. Grok showed the lowest parsability failure rate and therefore represented the top tier. GPT-4o and Mistral formed a second tier with very similar reliability, followed closely by Claude. DeepSeek and Gemini made up a third tier and displayed noticeably higher frequencies of structural faults. These results are illustrated in Fig. 5.

In addition to AST-based parsing, we introduced a completion integrity check on the model-generated code to detect cases where the output was incomplete. Specifically, we verified that each response returned a complete function enclosed within the requested ```` ```python ... ``` ```` delimiters and that generation was not prematurely truncated. Outputs that were cut off, missing closing delimiters, or otherwise incomplete were treated as completion-related failures and counted as parsability failures, since such outputs cannot be reliably parsed, integrated, or executed. This additional check identified 74 completion-related failures for Claude, 21 for Mistral, 5 for DeepSeek, 1 for Gemini, 1 for GPT-4o, and 0 for Grok. **All such completion-related failures are included in the parsability failure rates reported in** Fig. 5**.** These cases reflect differences in output stability rather than repair quality.

To better understand the nature of parsability failures detected during AST-based validation, we manually reviewed a sample of 200 refactorings that were marked as parsing failures. Most issues were attributable to common generation mistakes such as incomplete or unterminated code blocks, malformed f-strings, and unclosed delimiters. We also observed typical syntactic violations, including missing indented blocks after control statements or invalid operations such as `result.where(col < 5) *= 2`, which Python rejects as an illegal assignment target during parsing. These problems were confined to surface-level syntax, indicating that the majority of failures stem from minor structural inaccuracies rather than deeper issues in the transformation process.

To further assess the correctness of the LLM-generated repairs on this filtered set of functions (22,259), we adopt three complementary static metrics that capture different aspects of repair effectiveness. These metrics evaluate whether a repair resolves the originally reported issues, avoids introducing new static violations, and achieves full static correctness. All metrics are computed over the full set of evaluated functions. Repairs that fail AST-based parsability validation are treated as unsuccessful, since static analysis cannot be reliably performed on syntactically invalid code.

Static Repair Success Rate (SRSR) measures the proportion of functions for which the generated repair is a *complete repair*. A repair is considered complete if it passes AST-based parsability validation, eliminates all originally reported SonarQube rule identifiers, and introduces no new rule identifiers. It is defined as:

$$\text{SRSR} = \frac{\#\text{ complete repairs}}{\#\text{ total functions}} \times 100\%$$

Original Issue Resolution Rate (OIRR) measures the proportion of functions for which the generated repair removes all *originally reported SonarQube rule identifiers*. A repair is considered resolved if it passes AST-based parsability validation and none of the rule identifiers present in the original function remain present after repair, regardless of whether new rule identifiers are introduced. It is defined as:

$$\text{OIRR} = \frac{\#\text{ resolved repairs}}{\#\text{ total functions}} \times 100\%$$

Static Safety Rate (SSR) measures the proportion of functions for which the generated repair is a *safe repair*. A repair is considered safe if it passes AST-based parsability validation and introduces no new SonarQube rule identifiers compared to the original function, regardless of whether the original issues are resolved. It is defined as:

$$\text{SSR} = \frac{\#\text{ safe repairs}}{\#\text{ total functions}} \times 100\%$$

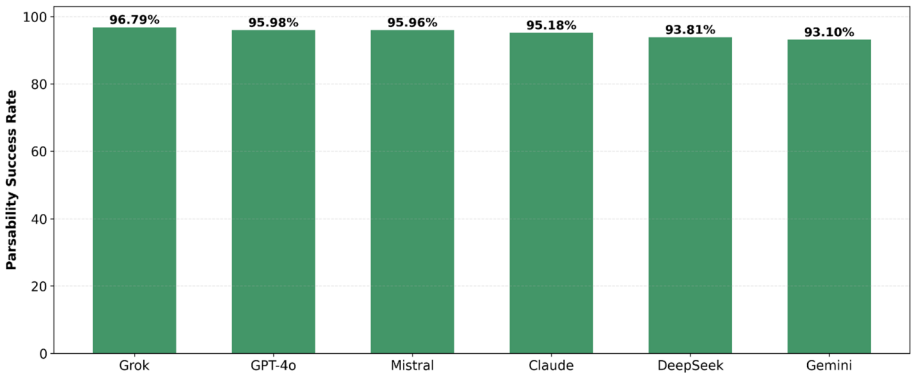

Fig. 5 Parsability success rate by model

Table 9 reports the three complementary static repair metrics, SRSR, OIRR, and SSR, across all evaluated models. Because SRSR is the most restrictive metric, requiring successful AST-based parsability, complete resolution of all originally reported SonarQube issues, and the absence of newly introduced violations, it serves as the primary basis for ranking. In contrast, OIRR and SSR capture partial aspects of repair quality by focusing separately on issue resolution and non-regression. Accordingly, the comparative ranking derived from SRSR is illustrated in Fig. 6, which provides a clearer visual representation of the performance differences between models.

Based on SRSR, the models separate into three clear performance tiers. The top tier consists of Grok (62.08%) and GPT-4o (59.13%), which achieve the highest rates of complete repairs. Grok leads across all three metrics, recording the strongest OIRR (66.70%) and the highest SSR (90.80%), indicating both effective elimination of original issues and strong resistance to introducing new violations. GPT-4o follows with consistently high scores in completeness (59.13%), resolution (62.89%), and non-regression (90.62%). Together, these two models demonstrate the most reliable ability to satisfy all static repair constraints simultaneously.

The second tier includes DeepSeek (58.40%), Claude (58.15%), and Mistral (56.68%). Their SRSR values fall within a narrow interval of less than two percentage points, indicating closely comparable levels of complete repair capability. Within this group, Claude records a slightly higher OIRR (62.48%) than DeepSeek (61.66%) and Mistral (60.60%), suggesting marginally stronger effectiveness at resolving original issues. In terms of SSR, Mistral reaches 90.11%, slightly above Claude (89.51%) and DeepSeek (89.15%). Overall, these three models show balanced and competitive performance, though they remain just below the top tier under the strict SRSR criterion.

The third tier is represented solely by Gemini (50.89%), notably lower than the other models. Although its OIRR (59.01%) remains within the general range of the second tier, its SSR (81.11%) is substantially lower than all other models. This lower non-regression rate indicates a higher tendency to introduce new static violations, which limits its ability to achieve complete, rule-compliant repairs. Consequently, Gemini ranks last under the most restrictive evaluation criterion, despite demonstrating a moderate capacity to resolve original issues.

In summary, this subsection evaluated model performance at the function level using three complementary static metrics that capture completeness, issue resolution, and safety. By analyzing SRSR, OIRR, and SSR jointly, we distinguished between fully correct repairs, partial resolutions of original issues, and non-regressive transformations. The results show clear performance tiers under the strict SRSR criterion, while also revealing differences in how consistently models eliminate original violations and avoid introducing new ones. However, this analysis remains confined to static properties and does not yet examine the specific rule categories affected or the semantic implications of the generated changes. These aspects are explored in the subsequent subsection.

Table 9 Static repair effectiveness metrics across LLMs (bold values indicate the best observed result on each column)

Model	SRSR	OIRR	SSR
Grok	**62.08%**	**66.70%**	**90.80%**
GPT-4o	59.13%	62.89%	90.62%
DeepSeek	58.40%	61.66%	89.15%
Claude	58.15%	62.48%	89.51%
Mistral	56.68%	60.60%	90.11%
Gemini	50.89%	59.01%	81.11%

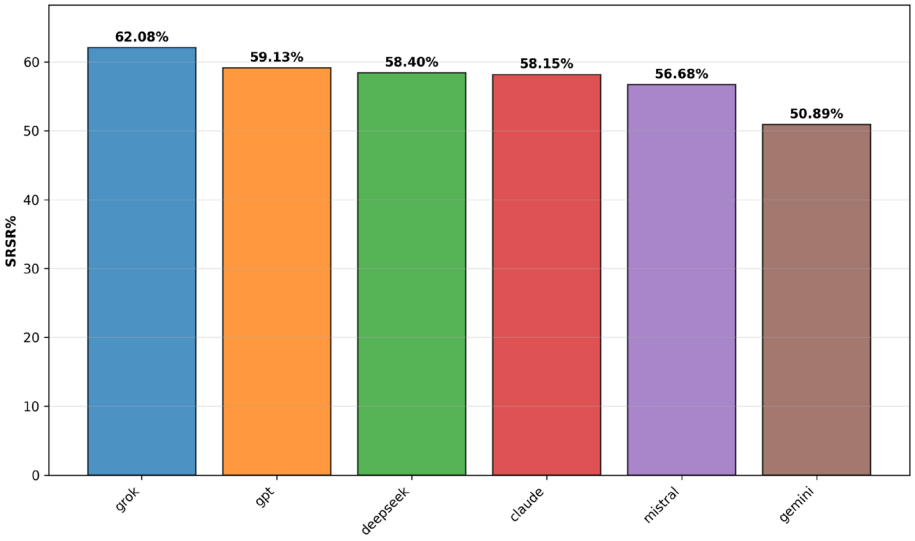

Fig. 6 Static Repair Success Rate (SRSR) by model

5.3 Issue-Level Repair Effectiveness

We continue the analysis by examining the same set of 22,259 functions considered in the previous subsection in order to provide an answer to RQ3. For each of these functions, we inspected the SonarQube issues originally reported before repair, then compared them with the issues remaining and any new issues introduced after applying each LLM's suggested fix. This detailed inspection aims to confirm that the reductions observed at the project and function levels correspond to meaningful improvements in code quality rather than superficial changes or offsetting errors. The post-repair SonarQube analysis was conducted under identical conditions for each model to ensure consistency and fairness in the evaluation. This finer-grained perspective also allows us to detect rule-specific patterns that may influence the interpretation of higher-level repair metrics.

This issue-level analysis focuses specifically on the SonarQube rule identifiers associated with the fixed and added issues, enabling us to assess not just how many issues were addressed but also which types of issues each model handled most effectively and where new problems might arise. As we mentioned in Section 2.1, SonarQube issues can be classified by the rule that triggers them, which provides a structured way to analyze the nature of the repairs. At this stage, we set the groundwork for understanding how the repairs interact with the specific rule categories and whether the observed patterns align with the models' strengths suggested by the earlier function-level results. Overall, the 22,259 functions in this subset were originally marked by one or multiple rules drawn from a set of 104 distinct SonarQube rule identifiers. Further details on the observed distributions and trends will follow as the analysis progresses.

To illustrate the issue-level behavior of the models in more detail, we selected the 10 most frequently occurring Python-specific SonarQube rules observed in the original, pre-repair version of the 22,259 functions. This selection highlights the most common and impactful types of issues present before any model intervention, providing a representative

overview of the recurring code-quality problems addressed by the models. As shown in previous studies, a relatively small subset of SonarQube rules is responsible for the majority of detected issues, and focusing on the ten most frequent rules is a well-established approach for summarizing such distributions (Baldassarre et al. 2020; Molnar and Motogna 2020).

Table 10 lists these 10 rules along with their descriptions. These rules account for a substantial proportion of the total violations in the function-level dataset considered in this issue-level analysis, rather than in the global SonarQube totals reported earlier in the paper. Table 11 presents, for each of these rules and each model, the number of remaining violations, including those not fixed and any new ones introduced by the model, alongside the percentage reduction compared to the original violations. All counts reported in this table are computed on the validated set of 22,259 function-level issue groups described in Section 5.2, and therefore differ from the global SonarQube totals presented earlier. This provides a detailed view of how effectively each model addressed the most frequent issues within the evaluated function-level scope and to what extent they reduced the original violations.

As presented in Table 11, our dataset is imbalanced, with certain SonarQube rule violations occurring far more frequently than others. The most frequent rules are S117, S1192, and S3776, which account for 9,158 (28.45%), 6,595 (20.49%), and 2,436 (7.57%) of the total 32,186 recorded issues, respectively. Together, these three rules comprise 56.51% of all violations, clearly illustrating the significant skew in the distribution of rule violations. Imbalance in software quality datasets is a well known phenomenon observed in many contexts, and several studies have proposed methods to mitigate its effects. As shown in Rathore et al. (2022), generative oversampling techniques have been applied to address imbalance in software fault prediction tasks, emphasizing that uneven distributions of issues are a common characteristic of software engineering data.

Although the models evaluated in this study are not trained on the dataset, such an imbalance still influences the interpretation of results. Aggregate metrics such as the overall issue reduction rate or function-level static repair metrics (e.g., SRSR, OIRR, and SSR) are dominated by the most frequent rule categories, meaning that high performance on common and often simpler rule types (e.g., naming conventions or string duplication) can disproportionately increase the overall effectiveness. Consequently, this skews the evaluation toward these prevalent issue types and may underrepresent model behavior on less frequent but more semantically complex rules, limiting the generality of the conclusions.

The issue-level analysis presented above is based on the validated subset of functions described in Section 5.2 and therefore differs from the global SonarQube totals reported earlier in Section 4.1, which refer to all detected issues across the analyzed repositories. Table 12 also relies on this validated subset and reports repair outcomes only for functions where extraction, repair generation, and validation were successfully completed. Using this subset, we aggregate repair outcomes by SonarQube issue type to examine how model performance varies across broader categories of issues.

As expected, code smells exhibit the highest reduction rates across all models, ranging from 42.9% (Gemini) to 60.6% (Grok). These results align with the predominantly structural and stylistic nature of code smells, which can often be addressed through localized refactorings. Bug-level issues, which are less frequent and typically require deeper semantic reasoning, show consistently lower but still substantial reduction rates, between 31.8% (Gemini) and 49.4% (Grok). This suggests that LLMs are not limited to correcting frequent

Table 10 Top 10 SonarQube Python code quality rules

Rule ID	Rule Title
S117	Local variables and parameters should comply with a naming convention
S1192	String literals should not be duplicated
S3776	Cognitive Complexity of functions should not be too high
S1172	Unused function parameters should be removed
S1135	Track uses of TODO tags
S5806	Builtins should not be shadowed by local variables
S1186	Functions and methods should not be empty
S1542	Function names should comply with a naming convention
S1481	Unused local variables should be removed
S112	"Exception" and "BaseException" should not be raised

Table 11 Reduction percentages of open issues across AI models for the top 10 Python code quality rules (bold values indicate the best observed result in each row)

Rule	Original	GPT-4o	Gemini	Grok	Claude	Mistral	DeepSeek
S117	9158	2530	3651	**2416**	2945	2613	2579
		-72.4%	-60.1%	**-73.6%**	-67.8%	-71.5%	-71.8%
S1192	6595	5326	5510	**5224**	5284	5286	5298
		-19.2%	-16.5%	**-20.8%**	-19.9%	-19.8%	-19.7%
S3776	2436	1206	1025	**574**	757	1422	1089
		-50.5%	-57.9%	**-76.4%**	-68.9%	-41.6%	-55.3%
S1172	2136	390	1169	476	**363**	384	506
		-81.7%	-45.3%	-77.7%	**-83.0%**	-82.0%	-76.3%
S1135	1593	330	500	**269**	586	620	374
		-79.3%	-68.6%	**-83.1%**	-63.2%	-61.1%	-76.5%
S1186	886	106	578	**74**	89	114	125
		-88.0%	-34.8%	**-91.6%**	-90.0%	-87.1%	-85.9%
S5806	884	359	352	**345**	358	357	367
		-59.4%	-60.2%	**-61.0%**	-59.5%	-59.6%	-58.5%
S1542	844	245	282	**238**	246	246	244
		-71.0%	-66.6%	**-71.8%**	-70.9%	-70.9%	-71.1%
S1481	763	447	579	548	**337**	426	363
		-41.4%	-24.1%	-28.2%	**-55.8%**	-44.2%	-52.4%
S112	698	69	75	63	**61**	62	68
		-90.1%	-89.3%	-91.0%	**-91.3%**	-91.1%	-90.3%

stylistic patterns, but are also capable of repairing a meaningful portion of semantically grounded defects.

Vulnerability-related results should be interpreted with caution. Although relatively high reduction percentages are observed, ranging from 45.8% (Gemini) to 70.8% (DeepSeek), the total number of vulnerabilities in the dataset is very small (24 issues), which precludes statistically robust conclusions. Consequently, the vulnerability figures are reported for completeness but are not used to draw comparative claims between models. Overall, this issue-type-based analysis complements the rule-level evaluation by providing a broader view of LLM repair capabilities beyond the dominant class of code smells.

Both Tables 10 and 11 highlight a clear imbalance in the distribution of rule violations. In particular, the top three rules, S117, S1192, and S3776, account for more than half of all detected issues, which explains their strong influence on aggregate repair metrics. These rules correspond to enforcing naming conventions, avoiding duplicated string literals, and limiting cognitive complexity, respectively, and are considered important for achieving maintainable and understandable code. As shown in Feitelson et al. (2020), naming is a cognitively demanding activity for developers, which makes violations of naming conventions a common occurrence detected by static analysis tools. Similarly, Gergel et al. (2011) shows that adherence to conventions and avoidance of duplication are associated with improved maintainability. Cognitive complexity has been empirically validated in Muñoz Barón et al. (2020) as a factor that increases the effort required to comprehend and modify code.

Table 11 shows that none of the evaluated models achieves superior performance across all rules, indicating that each model has distinct strengths and trade-offs. Grok attains the highest overall rank and delivers substantial reductions on key rules, standing out particularly on S117 naming conventions, where its reduction achieves 73.6%. However, it performs less strongly on S112, raising Exception or BaseException, where Claude achieves the highest reduction. GPT-4o and Claude also perform strongly overall, with Claude showing particular strength on rules like S112 and S1172, achieving 91.3% and 83.0% reductions respectively. These findings suggest that while all models are capable of addressing frequent and impactful issues to a significant degree, their effectiveness varies considerably depending on the rule category, which highlights the importance of tailoring LLM use to the specific quality concerns of a project.

A similar pattern emerges when examining the performance of the lower-ranked models. Although Gemini generally lags behind the others in overall performance, it does not come in last in every scenario. For example, on S3776, which measures cognitive complexity, Gemini achieves a reduction of 57.9%, better than GPT-4o (50.5%), Mistral (41.6%) and DeepSeek (55.3%), but still lower than the top performer, Grok, which achieves 76.4%. These observations underscore that no single model can yet fully address the breadth of SonarQube rule violations optimally, and that combining models or selecting the most appropriate one based on the predominant issue types could lead to more effective automated repairs.

While most frequent rules are effectively handled through localized refactorings, certain rule types expose structural limitations of a function-scoped repair strategy. Among the top ten most frequent SonarQube rules analyzed, S1192 (String literals should not be duplicated) stands out as the only one for which all models achieved relatively low reduction rates, consistently below 50%. This outcome likely stems from the nature of the violation and its misalignment with our function-level repair approach. Duplicated string literals can

Table 12 Reduction percentages of open issues across AI models by issue type (bold values indicate the best observed result in each row)

Issue Type	Original	GPT-4o	Gemini	Grok	Claude	Mistral	DeepSeek
Code Smell	30789	12877	17580	**12126**	13181	13659	12839
		-58.2%	-42.9%	**-60.6%**	-57.2%	-55.6%	-58.3%
Bug	1373	736	936	**695**	741	792	742
		-46.4%	-31.8%	**-49.4%**	-46.0%	-42.3%	-46.0%
Vulnerability	24	9	13	9	9	9	**7**
		-62.5%	-45.8%	-62.5%	-62.5%	-62.5%	**-70.8%**

be distributed across multiple functions or outside function bodies within the same file, so eliminating them within a single function does not necessarily resolve the overall issue. Even if an LLM refactors a function to remove repeated strings locally, SonarQube may continue to flag the file if the same literal appears in other functions, global variables, or class definitions. This highlights a limitation of function-scoped refactoring when addressing problems that inherently depend on file-level or module-level context.

The low reduction rates observed for rule S1192 (String literals should not be duplicated) prompted us to examine whether other SonarQube rules are similarly unsuitable for function-level refactoring. We reviewed the rules by initially querying the GPT-4o API with: "Which SonarQube rules cannot be fixed by editing a single function because the issue involves multiple functions, or the fix requires changes across more than one function?" GPT-4o recommended several rules as potentially requiring broader, cross-function, or file-level context, among which S1192 and S930 were included.

To assess these recommendations, we subsequently performed a comprehensive manual review of the Python rules observed in the dataset. Each rule was examined with respect to whether its resolution inherently depends on file-level or cross-function context, which would make it unsuitable for isolated function-level refactoring. This manual analysis independently confirmed that S1192 and S930 fundamentally and consistently require broader contextual information. While other rules suggested by GPT-4o, such as S2638 (Method overrides should not change contracts, 235 original issues) and S4144 (Functions should not have identical implementations, 135 original issues), may benefit from wider context, they either represented a small fraction of the total issue volume or could still be partially addressed at the function level and were therefore retained in the evaluation.

Across all analyzed rules, S1192 and S930 were the only ones for which both the GPT-4o recommendations and our independent manual review converged in concluding that function-scoped repair is structurally insufficient. S1192 accounted for 6,595 original issues, representing the second largest rule category in the dataset, and its resolution inherently requires cross-file knowledge of duplicate occurrences. Because duplicated string literals may appear in multiple functions or outside function bodies, isolated function-level refactoring cannot reliably eliminate the violation at the file level. This structural mismatch explains the consistently low reduction rates observed for S1192 across models and confirms that the limitation lies in the repair granularity rather than in model capability.

Rule S930, although comprising only 24 original issues, exhibited severe negative reductions across all six models after repair, further confirming the same structural limitation. Following LLM-generated refactorings, the number of S930 violations increased to 334 for Claude, 242 for DeepSeek, 199 for Gemini, 373 for GPT-4o, 379 for Grok, and 283 for Mistral. Manual inspection showed that these new violations were typically introduced when unused parameters were removed to satisfy S1172, while corresponding function calls elsewhere in the codebase were not updated. This created inconsistencies between function definitions and their callers, producing cross-functional mismatches. Consequently, we excluded S1192 and S930 from the evaluation dataset to ensure that the function-level repair assessment reflects rule categories compatible with the chosen repair scope.

These observations regarding rules `S1192` and `S930` reveal inherent limitations of our function-level refactoring approach, suggesting that the low reduction rates and increased violations were not solely due to the LLMs' capabilities but to a mismatch between the rules' nature and a function-scoped strategy. Because these limitations directly affect the

assessment of function-level static correctness, they also impact the interpretation of the static repair metrics introduced in the previous subsection, particularly the Static Repair Success Rate (SRSR). Including rules that fundamentally require cross-function or file-level context biases SRSR, OIRR, and SSR downward by penalizing models for cases that cannot be resolved under a function-scoped repair strategy.

To mitigate this effect, we repeated the function-level static evaluation after excluding functions affected by rules `S1192` or `S930`. Specifically, functions were excluded if either rule appeared in the original code or in any of the LLM-generated refactorings. From the subset of 22,259 function groups discussed above, we identified 5,147 functions that met this criterion and excluded them from the evaluation. This resulted in a final set of 17,112 function-level repairs per model used for the revised static repair analysis.

Among the excluded cases, most of the removed functions were affected only by rule `S1192`. These cases provided limited analytical value, as violations of this rule cannot be effectively resolved through function-scoped refactoring and therefore do not meaningfully reflect the LLMs' repair capabilities. As a result, the refactored versions remained effectively identical to the originals from a code quality perspective, with no real opportunity for improvement under a function-level strategy. Rule `S930` appeared in only 295 functions overall, making its impact on the overall evaluation comparatively small. Consequently, this filtering step was methodologically justified, as it removed cases that could not be effectively addressed within our function-based refactoring approach, while retaining those for which this strategy was appropriate and informative.

After applying the filtering that excludes functions affected by rules `S1192` and `S930`, all evaluated models exhibit substantial and consistent improvements across Static Repair Success Rate (SRSR), Original Issue Resolution Rate (OIRR), and Static Safety Rate (SSR). In particular, SRSR increases by approximately 10–13 percentage points for every model, indicating that a significant portion of previously unsuccessful repairs was attributable to rule categories that are structurally incompatible with function-scoped refactoring rather than to deficiencies in the models' repair capabilities.

As shown in Table 13, Grok remains the top-performing model after filtering, achieving an SRSR of 74.21%, followed by GPT-4o at 70.79%. DeepSeek and Claude reach comparable levels of complete static repair at 69.79% and 69.47%, respectively. Mistral and Gemini also show improvements, although they remain comparatively less effective under the strict SRSR criterion, with 67.61% and 60.92%, respectively. Importantly, the relative ranking of models remains unchanged after filtering, demonstrating that the original ordering was not driven by these rule categories. The magnitude of the SRSR increase before and after filtering is illustrated in Fig. 7.

The observed improvements across all three metrics confirm that rules `S1192` and `S930` systematically penalized models under strict static correctness criteria despite being unsuitable for resolution at the function level. By excluding these cases, the revised metrics provide a more accurate, unbiased, and methodologically sound assessment of each model's ability to generate syntactically valid, rule-compliant, and non-regressive repairs within the intended scope of the repair strategy. This filtering step therefore strengthens the internal validity of the function-level evaluation and clarifies that the remaining performance differences reflect genuine variation in repair effectiveness rather than artifacts of rule granularity mismatch.

Table 13 Static repair effectiveness metrics across LLMs after excluding rules S1192 and S930 (bold values indicate the best observed result on each column)

Model	SRSR	OIRR	SSR
Grok	**74.21**	**79.07**	**92.12**
GPT-4o	70.79	74.78	91.62
DeepSeek	69.79	73.20	89.81
Claude	69.47	74.15	90.09
Mistral	67.61	71.83	91.02
Gemini	60.92	69.96	82.00

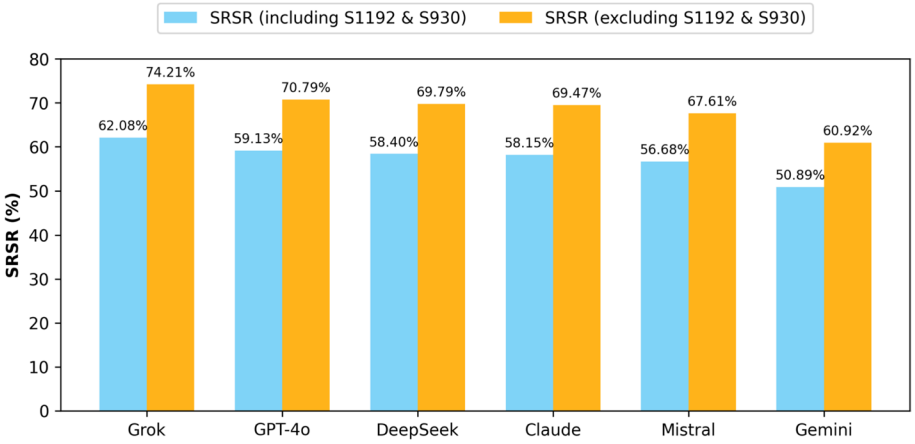

Fig. 7 Comparison of SRSR values before and after excluding rules S1192 and S930

The issue-level analysis shows that model performance depends strongly on the nature of the underlying SonarQube rules. Localized rules, such as naming or unused variable detection, were handled effectively, whereas broader rules like duplication or structural dependencies remained difficult to address. This alignment between issue-level limitations and function-level correctness metrics reinforces the validity of the adjusted evaluation.

5.4 Test-Suite-Based Validation Results

Following the test-suite-based validation protocol described in Section 4.4, this subsection addresses **RQ4** by introducing an execution-based evaluation applied to the filtered set of 17,112 function-level repairs identified in Section 5.3, after excluding cases associated with rules `S1192` and `S930`. From this set, a random sample of **139 original functions per project** was selected for evaluation, and the same sampled set was used across all six LLMs to assess whether LLM-generated repairs preserve the observable runtime behavior of the original code, as captured by the projects' test suites. The only exception is the *Requests* project, which contains fewer eligible modified functions; therefore, all **56 available functions** were included. This sampling strategy produced a total of **1,863 candidate functions** for execution-based validation.

We summarize test-suite outcomes using the **plausible correctness rate (PCR)**. A function-level repair is considered plausibly correct if the summary of test outcomes after applying the modification is identical to the baseline execution on the original, unmodi-

fied codebase, that is, the same number of passed tests, failed tests, errors, xpassed, and xfailed tests is observed. This definition adopts a strict equivalence criterion and intentionally avoids partial credit in order to minimize ambiguity in the interpretation of execution-based results.

In this execution-based setup, AST validation was not applied before test execution, since AST parsing does not capture runtime-breaking issues such as missing imports. Instead, we performed a lightweight string-based sanity check to detect incomplete outputs, including unbalanced code fences, unmatched parentheses, or truncated function bodies. Such cases were treated as execution errors and counted as implausible repairs in the PCR calculation.

All test outcomes are evaluated relative to project-specific baseline executions, and each function-level modification is tested in isolation following the sampling strategy described in Section 4.4. This design ensures that observed differences can be causally attributed to individual repairs rather than interactions between multiple changes. In the following analysis, we report and interpret execution-based results across projects and models. Due to hardware constraints, environment incompatibilities, and long execution times (approximately two days per project on average), some projects (keras, pandas, openpilot, zulip, and jumpserver) execute test suites that exclude specific categories of tests, such as those marked as *slow*, *network*, or other resource-intensive markers. In the case of *ansible*, due to execution constraints, only unit tests were executed.

Table 14 reports the plausible correctness rate for function-level repairs across the subset of projects with stable test-suite behavior. Only **14 out of 20** analyzed projects satisfied the stability requirements described in Section 4.4, namely reproducible baseline executions with consistent test outcome summaries. Consequently, the PCR results reported in Table 14 are computed only on the sampled functions belonging to these stable projects.

The remaining six projects could not be used in the execution-based validation because repeated baseline runs on the original, unmodified code produced inconsistent numbers of passed and failed tests. In other words, even the original version without any refactoring triggered fluctuating outcomes. This instability was caused by factors such as reliance on random number generation without fixed seeds, hardware-dependent behavior, partial or state-dependent database initialization requirements, and dependencies on external services or cloud resources. For example, yt-dlp depends on online platforms such as YouTube, where rate limiting and network variability affect test determinism.

For the 14 stable projects, the reported values indicate the proportion of sampled repairs for which the execution summary of the test suite remained identical to the baseline, providing an execution-based view of behavioral preservation. The observed variation across projects reflects differences in test coverage, architecture, and sensitivity to localized code changes. Across most included projects, all evaluated models achieve non-trivial plausible correctness rates, suggesting that a considerable fraction of function-level repairs do not introduce observable behavioral regressions under stable test conditions.

Although the 14 retained projects satisfied the stability requirements in terms of reproducible baseline executions, their test suites are not uniform in size or execution completeness. As shown in Table 15, several projects exhibit skipped tests according to the last observed execution logs, with skip rates ranging from negligible values (e.g., 0.05% in Zulip) to more substantial proportions (e.g., 12.88% in Fairseq). While skipped tests do not invalidate the execution-based validation, they reduce the effective runtime coverage and may limit the detection of certain behavioral regressions. As mentioned before, some

Docker configurations may exclude tests before collection, so they may not be reflected in the skipped counts.

In addition, for one project, Jumpserver, we identified only a single executable test in its default test suite during our analysis. Although this test appeared stable and reproducible, its extremely limited scope significantly constrains the strength of behavioral conclusions that can be drawn for this project. Consequently, the execution-based results for Jumpserver should be interpreted cautiously. More generally, the results across projects should be considered in light of differences in test-suite size, skipped-test rates, and coverage, rather than as uniformly strong behavioral guarantees.

To compare the models from a ranking perspective, we apply the Friedman test across the same 14 projects with stable test-suite behavior. In this setting, each project is treated as a block, and the models are ranked within each project according to their Plausible Correctness Rate (PCR). The average ranks across projects are then computed and reported in Table 16. The ranking shows that Grok achieves the best overall position, followed closely by DeepSeek, while Mistral and GPT-4o occupy intermediate positions, and Claude and Gemini appear in the lower ranks.

Complementing this rank-based comparison, descriptive statistics of PCR values across the projects are summarized in Table 17. Across models, mean PCR values range between 71.65% and 79.43%, indicating that a substantial fraction of function-level repairs do not introduce observable behavioral regressions when evaluated under stable test-suite conditions. Grok and DeepSeek achieve the highest mean PCR, both at 79.43%, followed by Mistral at 78.68% and GPT-4o at 78.30%. Claude reaches 77.94%, while Gemini records the lowest mean PCR at 71.65%.

Standard deviations reported in Table 17 are comparable across models, ranging from 12.24 to 15.43, reflecting moderate variability driven by project-specific factors such as test coverage and sensitivity to localized changes. DeepSeek shows the lowest variability (12.24), indicating the most consistent execution-based behavior across projects, while Gemini exhibits the highest variability (15.43). Overall, these statistics reinforce the conclusion that while all models can generate behavior-preserving repairs in many cases,

Table 14 Plausible Correctness Rate (PCR) across projects with stable test-suite behavior (bold values indicate the best observed result in each row)

Project	Claude	GPT-4o	Gemini	Grok	Mistral	DeepSeek
Ansible	94.96%	**97.84%**	94.24%	96.40%	96.40%	94.96%
Celery	**66.19%**	61.87%	56.83%	63.31%	61.87%	64.75%
Django RF	52.52%	58.99%	51.80%	56.12%	55.40%	**62.59%**
ERPNext	87.77%	87.05%	**89.93%**	87.77%	**89.93%**	88.49%
Fairseq	84.89%	**88.49%**	84.89%	**88.49%**	86.33%	87.77%
JumpServer	**94.24%**	**94.24%**	90.65%	93.53%	93.53%	93.53%
MMDet	83.45%	79.86%	62.59%	**84.89%**	84.17%	**84.89%**
OpenPilot	90.65%	89.93%	87.05%	91.37%	91.37%	**92.81%**
Pandas	72.66%	72.66%	62.59%	**79.14%**	73.38%	71.94%
Requests	80.36%	83.93%	75.00%	**87.50%**	85.71%	83.93%
Scrapy	58.99%	**61.15%**	53.24%	58.27%	57.55%	59.71%
SD WebUI	**84.89%**	83.45%	75.54%	84.17%	83.45%	82.01%
Tornado	64.75%	62.59%	55.40%	64.03%	**66.19%**	**66.19%**
Zulip	74.82%	74.10%	63.31%	76.98%	76.26%	**78.42%**

Table 15 Test-suite characteristics for projects with stable execution behavior

Project	Total Tests	Skipped (%)
Ansible	2048	5 (0.24%)
Celery	3383	26 (0.77%)
Django RF	1558	73 (4.69%)
ERPNext	1962	0 (0.00%)
Fairseq	458	59 (12.88%)
Jumpserver	1	0 (0.00%)
MMDet	828	22 (2.66%)
OpenPilot	657	65 (9.89%)
Pandas	231384	6242 (2.70%)
Requests	608	15 (2.47%)
Scrapy	3457	187 (5.41%)
SD WebUI	33	0 (0.00%)
Tornado	4880	20 (0.41%)
Zulip	4050	2 (0.05%)

Table 16 Average model ranks from the Friedman test for Plausible Correctness Rate (PCR) across projects

Model	Rank	Avg. Rank
Grok	1	2.6071
DeepSeek	2	2.7143
Mistral	3	3.0714
GPT-4o	4	3.4286
Claude	5	3.5357
Gemini	6	5.6429

notable differences exist in both reliability and stability across execution-stable real-world codebases.

Overall, the execution-based validation indicates that many LLM-generated repairs preserve the observable runtime behavior of the original programs under stable test conditions. However, because test-suite evaluation depends on coverage and deterministic environments, it cannot fully assess all repaired functions. To complement this runtime perspective, the next subsection introduces a semantic consistency estimation based on automated comparison of original and refactored code.

5.5 Estimated Semantic Equivalence via LLMs

This subsection further addresses **RQ4** by introducing a complementary, estimation based assessment of semantic equivalence applied to the complete filtered set of 17,112 function-level repairs identified in Section 5.3. While the execution-based validation presented in Section 5.4 relies on concrete program execution and test outcomes, its applicability is limited to projects with stable and reproducible test suites, and test coverage may not exercise all repaired functions.

To complement this limitation, we introduce a semantic consistency evaluation that provides a scalable approximation of functional equivalence at the function level. The goal of this evaluation is not to replace execution-based validation, but to estimate whether refactored functions preserve the intended behavior of the original code in scenarios where runtime evidence is unavailable or incomplete.

Table 17 Descriptive statistics of Plausible Correctness Rate (PCR) across projects

Model	Mean	Std	Min	25%	Median	75%	Max
Grok	79.43	13.57	56.12	67.27	84.53	88.31	96.40
DeepSeek	79.43	12.24	59.71	67.63	82.97	88.31	94.96
Mistral	78.68	13.71	55.40	67.99	83.81	89.03	96.40
GPT-4o	78.30	13.14	59.00	65.11	81.65	88.13	97.84
Claude	77.94	13.31	52.52	67.81	81.91	87.05	94.96
Gemini	71.65	15.43	51.80	58.27	69.15	86.51	94.24

In this evaluation, we employed a large language model, **GPT-4o**, as an automated *semantic equivalence estimator*. The choice of GPT-4o is motivated by prior work that proved its applicability to software engineering classification and code analysis tasks, as proved in Ghammam et al. (2025), as well as studies that showed the suitability of large language models for automated GitHub issue classification in software engineering contexts, as shown in Heo and Lee (2025).

For each repaired function, the model was provided with the original and the modified version and prompted to assess, as a heuristic judgment, whether the change preserves the intended behavior of the original code. Each comparison was framed as a binary decision task, where the model responded with "YES" if the two versions were estimated to be functionally equivalent and "NO" otherwise.

This process yields an **estimated semantic equivalence rate**, defined as the proportion of repairs judged to preserve functionality. Importantly, this evaluation should be interpreted as an approximation of semantic equivalence rather than a definitive oracle, as it relies on probabilistic model judgment rather than on program execution.

Table 18 reports the estimated semantic equivalence rate for each evaluated LLM on the filtered set of functions, after excluding cases associated with rules S1192 and S930. The results range from 66.01% to 74.25%, indicating that for a majority of repairs the generated changes are estimated to preserve the original behavior. Grok achieves the highest semantic equivalence rate, followed by DeepSeek and GPT-4o, while Gemini exhibits the lowest estimated preservation rate among the evaluated models.

Although semantic equivalence estimation and test-suite validation rely on different signals, their model rankings are largely consistent. The Friedman ranking based on the Plausible Correctness Rate (PCR) places Grok first, followed by DeepSeek, Mistral, GPT-4o, Claude, and Gemini (Table 16). A very similar ordering appears in the LLM-based semantic equivalence results (Table 18), where Grok and DeepSeek again lead, while Gemini ranks last. The main difference is the relative position of GPT-4o and Mistral, which swap places depending on the evaluation metric.

To further investigate the relationship between these two evaluation approaches, we measured their agreement on the subset of 1,863 functions that were evaluated using test-suite execution. For each function, six repairs were generated, one by each evaluated LLM, resulting in six repair evaluations per function. For every repair, we compared the binary decision produced by the LLM-based semantic estimator (whether the refactoring preserves functionality) with the outcome observed through the test-suite validation protocol. An agreement occurs when both methods reach the same conclusion regarding whether the refactoring preserves or changes the behavior of the original function.

Table 18 Estimated semantic equivalence rate (LLM-based) for function-level repairs on the filtered dataset

Model	Semantic equivalence (%)
Grok	74.25
DeepSeek	72.75
GPT-4o	72.01
Mistral	71.52
Claude	69.74
Gemini	66.01

Across the resulting 11,178 repair evaluations (1,863 functions $\times$ 6 model-generated repairs), the two approaches produced identical outcomes in 7,962 cases, corresponding to an overall agreement rate of 71.23%. Among these agreements, 7,128 cases represent situations where both methods concluded that the repair preserves the original behavior, while the remaining agreements correspond to cases where both methods detected behavioral changes.

To further quantify the level of agreement between the two evaluation methods, we computed Cohen's kappa, a statistical measure commonly used to assess inter-rater agreement while accounting for agreement that may occur by chance. The resulting value of 0.158 indicates only slight agreement between the semantic estimation and the test-suite-based validation. This relatively low value suggests that the two approaches capture different aspects of behavioral correctness: test suites provide execution-based evidence limited by test coverage, whereas LLM-based semantic estimation evaluates functional intent at the code level. Nevertheless, the substantial number of cases in which both methods agree on preserved behavior provides additional evidence that many LLM-generated refactorings maintain the intended functionality of the original code.

The semantic equivalence estimation complements execution-based validation by extending the behavioral analysis to a larger set of repairs. Although the two approaches rely on different signals, their results consistently suggest that many LLM-generated refactorings preserve the intended behavior of the original code. Similar to the execution-based validation, minor differences in model rankings can be observed, particularly in the relative positions of GPT-4o and Mistral, which switch places depending on the evaluation metric. Together, these perspectives provide a broader view of behavioral correctness in LLM-assisted code repair.

6 Discussion

The results of this study, viewed through a multi-layer evaluation framework combining static, issue-level, execution-based, and semantic analyses, provide a nuanced understanding of the strengths and limitations of large language models in repairing SonarQube-reported issues. At the project level, LLMs consistently reduced static analysis findings across diverse, complex Python projects, demonstrating their potential as practical tools for automated code remediation. These reductions show that LLMs can scale improvements even in large, heterogeneous codebases. However, the magnitude of improvement varied by project, likely influenced by differences in structure, modularity, and adherence to best practices. While project-level metrics highlight the promise of LLMs in large-scale scenarios, they also obscure the quality and nature of individual fixes. Thus, these reductions should be

seen as encouraging signals of potential rather than definitive evidence of uniformly high-quality repairs throughout the codebase.

At the function level, the analysis further evaluated the correctness of LLM-generated repairs by their ability to produce syntactically valid, rule-compliant function-level fixes that eliminate all originally reported SonarQube issues without introducing new violations, as measured by the Static Repair Success Rate (SRSR). Grok and GPT-4o achieved the highest SRSR values, indicating they were most effective at producing repairs that were parsable and statically complete with respect to the reported quality problems. Their success suggests a stronger understanding of issues within the function's context. In contrast, Mistral and Gemini exhibited the lowest SRSR scores, though still above 50%, showing that all models contributed meaningfully but with varying reliability under strict static correctness constraints. Overall, this analysis highlights the models' capacity to make precise, meaningful corrections to localized issues, with clear differences in their ability to deliver fully resolved and non-regressive static repairs.

At the issue level, the analysis examined how effectively each model addressed the specific SonarQube rules behind the reported issues. No single model was best across all rules, and each had strengths in different areas. Grok excelled in naming and code complexity, while Claude handled improper exception usage well. Rules tied to localized concerns, such as naming standards, removing unused variables or parameters, avoiding empty functions, and simplifying complex code, were handled most successfully. These results show the models' ability to improve stylistic and maintainability aspects of functions while revealing limitations on rules requiring broader file-level or cross-function context. This suggests that choosing the right model depends on the specific quality issues in the project rather than expecting one model to excel universally.

In addition, behavioral validation adds an execution-aware dimension to the analysis, assessing whether LLM-generated repairs preserve observable runtime behavior, while the semantic layer complements this view by estimating whether the generated functions preserve the intended functionality of the original code. By extending the evaluation beyond static parsability validation and issue resolution, these layers clarify how often repairs remain faithful to the original behavior.

Across all models, roughly two-thirds of the analyzed fixes were estimated to preserve the original functionality based on both test-suite execution results and LLM-based semantic evaluation, with **Grok** achieving the highest overall results across both execution-based validation (based on mean PCR across projects) and semantic estimation, while **Gemini** consistently obtained the lowest scores under both evaluation strategies. This additional perspective reinforces that structural validity does not always imply semantic correctness and that both static and dynamic behavioral dimensions are essential for assessing real code quality improvements.

Taken together, the multi-layer evaluation framework demonstrates that large language models hold strong potential for improving software quality through automated repairs. At the project level, they achieve measurable reductions in static analysis findings across diverse Python codebases. At the function and issue levels, static correctness metrics show that LLMs can produce syntactically valid and rule-compliant repairs, with effectiveness varying across rule categories. Execution-based validation and semantic estimation further indicate that many repairs preserve observable behavior and intended functionality. Overall, these findings point to a promising direction where LLMs can enhance code quality when applied with careful model selection and context-aware evaluation.

7 Threats to Validity

We adopt the recommendations from Sjøberg and Bergersen (2023) and Verdecchia et al. (2023) to discuss construct, internal, and external validity. Regarding construct validity, our study assumes that reducing SonarQube-reported issues improves code quality. However, as noted by Lenarduzzi et al. (2020), blindly fixing all warnings may introduce unnecessary or harmful changes.

Some of the analyzed projects, such as scikit-learn, pandas, and Zulip, already integrate static analysis tools or linters, including Ruff and SonarCloud, the cloud version of SonarQube that relies on the same analysis rules. This does not eliminate reported issues, as addressing all findings requires developer time and prioritization, and some warnings may remain unresolved or correspond to false-positive cases. Consequently, SonarQube still identified a substantial number of issues in these projects, and the results should be interpreted as a conservative estimate of LLM effectiveness.

Another construct validity concern relates to the evaluation of functional preservation. In this study, behavioral correctness is supported by execution-based validation through project-specific test suites, which provide runtime evidence of whether LLM-generated repairs preserve observable behavior. However, this validation depends on test coverage and determinism; if certain behaviors are not exercised, semantic regressions may remain undetected. As mentioned earlier, some project configurations may also exclude specific categories of tests, further reducing the effective behavioral coverage.

In addition to execution-based validation, we employed GPT-4o as an automated semantic equivalence estimator to scale the analysis across all filtered function-level repairs. While this approach complements test-suite evaluation and extends coverage to scenarios where execution-based validation is limited, LLM-based judgments remain probabilistic and approximate rather than definitive. Moreover, recent work highlights broader construct validity risks in LLM-based software engineering studies, including closed-source model opacity, potential implicit data leakage, and limited traceability of model reasoning processes (Sallou et al. 2024). These factors should be considered when interpreting both the semantic evaluation and the overall conclusions.

Internal validity is influenced by the experimental setup. Even with a unified prompt template, different models may react differently to the same formatting or contextual cues, which can affect their outputs. Using public API endpoints also exposed our results to factors like server load or throttling, which may have caused some of the incomplete or malformed responses we classified as parsability failures during AST-based validation.

External validity is limited by our focus on popular, open-source, actively maintained Python projects. Other languages, such as Java, Ruby, or C++, and less mature or proprietary codebases may produce different outcomes. Performance observed via public APIs under shared conditions might also differ from that in controlled private deployments or offline settings.

8 Conclusions and Future Work

This study shows that large language models can assist in repairing SonarQube-identified code issues in diverse Python projects. By systematically comparing several state-of-the-art models, the study revealed that LLMs can substantially reduce the number of reported

issues, improve maintainability, and enhance code quality. At the same time, the results highlight that effectiveness varies across models and issue types, and that generated fixes sometimes introduce new problems or fail to integrate cleanly. These findings highlight the potential of LLMs in software maintenance, while also emphasizing the need for careful validation and human oversight.

Looking ahead, several promising directions emerge. Integrating LLM-dri-ven repairs into tools like VS Code or Cursor could deliver automated suggestions during development. Hybrid systems combining LLMs with rule-based tools may produce more reliable, context-aware fixes. Beyond benchmarking accuracy, future research can investigate why and under what contextual conditions LLMs succeed or fail to resolve code issues. A systematic ablation of prompt components, such as including or omitting file-level imports, neighboring functions, or project-specific coding conventions, can help identify which contextual elements are necessary and sufficient for effective remediation.

Future work can also include a qualitative review of a representative sample of repaired functions, supported by direct human analysis of selected patches. Such an examination would clarify how the generated changes affect readability, structure, and clarity, and would complement the quantitative results. This line of inquiry could deepen understanding of LLM behavior and guide more principled prompt designs for automated code repair.

Additional research may explore advanced prompt engineering strategies, including multi-step or iterative prompting approaches that address one issue at a time or continuously refine generated patches until convergence. Such methods may reveal deeper insights into model reasoning and improve the precision and consistency of automated repairs. Adaptive prompting aligned with project conventions and developer feedback can improve repair quality and acceptance. Using more real-world static analysis data could further enhance model performance. Finally, evaluating LLMs on additional languages, larger codebases, and more complex issues would help generalize these findings and broaden their applicability.

By building on these insights and addressing the current limitations, future research and practice can move closer to seamlessly embedding LLMs into everyday software development, supporting developers with intelligent, reliable, and context-sensitive maintenance assistance. Further investigation into dynamic prompting and interactive repair workflows could provide a more nuanced understanding of how LLMs adapt to feedback and incremental corrections, contributing to more stable and context-aware repair strategies.

Funding The authors received no financial support for the research or authorship of this article.

Data Availability The dataset containing the functions extracted from the repositories and the refactored functions generated by the LLMs, along with scripts for dataset creation (SonarQube analysis), calling the LLM models, generating the semantic analysis, and replacing the original functions with the refactored ones, as well as the Dockerfiles used for the projects, the scripts that orchestrate their execution, and the logs generated during the test-suite runs, is publicly available. The full replication package can be accessed at the following link: https://figshare.com/s/677f1da58367c9aa4d67https://doi.org/10.6084/m9.figshare.29605463

Declarations

Compliance with Ethical Standards The authors declare that they have complied with all ethical standards in the preparation of this work, including adherence to research integrity, avoidance of plagiarism, and respect for intellectual property rights.

Conflict of Interest/Competing Interests The authors declare that they have no conflict of interest or competing interests.

Ethical Approval In this study, no research involving human participants or animals was conducted by the authors.

Informed Consent Not applicable.

References

Abtahi SM, Azim A (2025) Augmenting large language models with static code analysis for automated code quality improvements. In: FORGE

Arora C, Sayeed AI, Licorish S, Wang F, Treude C (2024) Optimizing large language model hyperparameters for code generation. arXiv preprint arXiv:2408.10577

Baldassarre MT, Lenarduzzi V, Romano S, Saarimäki N (2020) On the diffuseness of technical debt items and accuracy of remediation time when using sonarqube. Inf Softw Technol 128:106377. https://doi.org/10.1016/j.infsof.2020.106377. https://www.sciencedirect.com/science/article/pii/S0950584919302113

Baltes S, Angermeir F, Arora C, Barón MM, Chen C, Böhme L, Calefato F, Ernst N, Falessi D, Fitzgerald B et al (2025) Guidelines for empirical studies in software engineering involving large language models. arXiv preprint arXiv:2508.15503

Basili VR, Caldiera G, Rombach HD (1994) The goal question metric approach. https://api.semanticscholar.org/CorpusID:13884048

Berabi B, Gronskiy A, Raychev V, Sivanrupan G, Chibotaru V, Vechev M (2024) Deepcode ai fix: Fixing security vulnerabilities with large language models. arXiv preprint arXiv:2402.13291

Bucaioni A, Gualandi G, Toma J (2025) Benchmarking large language models for autonomous run-time error repair: Toward self-healing software systems. In: EASE

Chen M, Tworek J, Jun H, Yuan Q, Pinto HPDO, Kaplan J, Edwards H, Burda Y, Joseph N, Brockman G, et al. (2021) Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374

Cihan U, Icoz A, Haratian V, Tuzun E (2025) Evaluating large language models for code review. arXiv preprint arXiv:2505.20206

De Carvalho HDP, Fagundes R, Santos W (2021) Extreme learning machine applied to software development effort estimation. IEEE Access 9:92676–92687

Feitelson DG, Mizrahi A, Noy N, Shabat AB, Eliyahu O, Sheffer R (2020) How developers choose names. IEEE Trans Software Eng 48(1):37–52

Gergel B, Stroulia E, Smit M, Hoover HJ (2011) Maintainability and source code conventions: An analysis of open source projects. Technical Report, Univ. of Alberta

Ghammam A, Rzig DE, Almukhtar M, Khalsi R, Hassan F, Kessentini M (2025) Build code needs maintenance too: A study on refactoring and technical debt in build systems. arXiv preprint arXiv:2504.01907

Gnieciak D, Szandala T (2025) Large language models versus static code analysis tools: A systematic benchmark for vulnerability detection. arXiv preprint arXiv:2508.04448

Heo J, Lee S (2025) A study on applying large language models to issue classification. In: 2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC), IEEE Computer Society, pp 1–11

Jaoua I, Sghaier OB, Sahraoui H (2025) Combining Large Language Models with Static Analyzers for Code Review Generation . In: 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), IEEE Computer Society, pp 174–186, https://doi.org/10.1109/MSR66628.2025.00038

Jiang J, Wang F, Shen J, Kim S, Kim S (2024) A survey on large language models for code generation. arXiv preprint arXiv:2406.00515

Jin M, Shahriar S, Tufano M, Shi X, Lu S, Sundaresan N, Svyatkovskiy A (2023) Inferfix: End-to-end program repair with llms. In: Proceedings of the 31st ACM joint european software engineering conference and symposium on the foundations of software engineering, pp 1646–1656

Lefever J, Cai Y, Cervantes H, Kazman R, Fang H (2021) On the lack of consensus among technical debt detection tools. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp 121–130. https://doi.org/10.1109/ICSE-SEIP52600.2021.00021

Lenarduzzi V, Lomio F, Huttunen H, Taibi D (2020) Are sonarqube rules inducing bugs? In: 2020 IEEE 27th international conference on software analysis, evolution and reengineering (SANER), IEEE, pp 501–511

Lenarduzzi V, Pecorelli F, Saarimaki N, Lujan S, Palomba F (2023) A critical comparison on six static analysis tools: Detection, agreement, and precision. J Syst Softw 198:111575

Li R, Allal LB, Zi Y, Muennighoff N, Kocetkov D, Mou C, Marone M, Akiki C, Li J, Chim J et al (2023) Starcoder: may the source be with you! arXiv preprint arXiv:2305.06161

Moldovan VA, Berciu LM, Patcas RD (2024) The python software quality dataset. In: 2024 50th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), IEEE, pp 395–398

Molnar AJ, Motogna S (2020) Long-term evaluation of technical debt in open-source software. In: Proceedings of the 14th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), Association for Computing Machinery, New York, NY, USA, ESEM '20. https://doi.org/10.1145/3382494.3410673

Muñoz Barón M, Wyrich M, Wagner S (2020) An empirical validation of cognitive complexity as a measure of source code understandability. In: Proceedings of the 14th ACM/IEEE international symposium on empirical software engineering and measurement (ESEM), pp 1–12

Nijkamp E, Pang B, Hayashi H, Tu L, Wang H, Zhou Y, Savarese S, Xiong C (2022) Codegen: An open large language model for code with multi-turn program synthesis. In: International Conference on Learning Representations. https://api.semanticscholar.org/CorpusID:252668917

Quezada Sarmiento P, Guaman D, Barba Guamán LR, Enciso L, Cabrera P (2017) Sonarqube as a tool to identify software metrics and technical debt in the source code through static analysis

Rathore SS, Chouhan SS, Jain DK, Vachhani AG (2022) Generative oversampling methods for handling imbalanced data in software fault prediction. IEEE Trans Reliab 71(2):747–762

Renze M (2024) The effect of sampling temperature on problem solving in large language models. Findings of the association for computational linguistics EMNLP 2024:7346–7356

Sallou J, Durieux T, Panichella A (2024) Breaking the silence: the threats of using llms in software engineering. In: Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results, pp 102–106

Seo M, Choi W, You M, Shin S (2025) Autopatch: Multi-agent framework for patching real-world cve vulnerabilities. arXiv preprint arXiv:2505.04195

Shivashankar K, Orucevic M, Kruke MM, Martini A (2025) Beacon-td: Classifying technical debt and its types across diverse software projects issues using transformers. J Syst Softw 226:112435

Simões IRdS, Venson E (2024) Evaluating source code quality with large language models: a comparative study. In: Proceedings of the XXIII brazilian symposium on software quality, pp 103–113

Sjøberg DI, Bergersen GR (2023) Improving the reporting of threats to construct validity. In: Proceedings of the 27th international conference on evaluation and assessment in software engineering, association for computing machinery, New York, NY, USA, EASE '23, pp 205–209. https://doi.org/10.1145/3593434.3593449

Solari M, Vegas S, Juristo N (2018) Content and structure of laboratory packages for software engineering experiments. Inf Softw Technol 97:64–79

Tamberg K, Bahsi H (2025) Harnessing large language models for software vulnerability detection: A comprehensive benchmarking study. IEEE Access pp 29698–29717

Vassallo C, Panichella S, Palomba F, Proksch S, Gall HC, Zaidman A (2020) How developers engage with static analysis tools in different contexts. Empir Softw Eng 25:1419–1457

Verdecchia R, Engström E, Lago P, Runeson P, Song Q (2023). Threats to validity in software engineering research: A critical reflection. https://doi.org/10.1016/j.infsof.2023.107329

Wadhwa N, Pradhan J, Sonwane A, Sahu SP, Natarajan N, Kanade A, Parthasarathy S, Rajamani S (2024) Core: Resolving code quality issues using llms. Proceed ACM Softw Eng 1(FSE):789–811

Wieringa R, Maiden N, Mead N, Rolland C (2006) Requirements engineering paper classification and evaluation criteria: a proposal and a discussion. Requirements Eng 11:102–107

Xia CS, Wei Y, Zhang L (2023) Automated program repair in the era of large pre-trained language models. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, pp 1482–1494

Yeboah J, Popoola S (2023) Efficacy of static analysis tools for software defect detection on open-source projects. In: 2023 International Conference on Computational Science and Computational Intelligence (CSCI), IEEE, pp 1588–1593

Zhang B, Liang P, Zhou X, Zhou X, Lo D, Feng Q, Li Z, Li L (2024) A comprehensive evaluation of parameter-efficient fine-tuning on code smell detection. arXiv preprint arXiv:2412.13801

Zhang Q, Fang C, Ma Y, Sun W, Chen Z (2023) A survey of learning-based automated program repair. ACM Trans Softw Eng Methodol 33(2):1–69